DATAFLOW SCENARIO BASED Q&A

BY - SHUBHAM WADEKAR

Copyright © Shubham Wadekar. All rights reserved.

This material is for personal use only. No part may be copied, shared, resold, or published without written permission. Unauthorized distribution is strictly prohibited and may result in action.

For permissions or licensing, contact: [shubham.p.wadekar@gmail.com]

Disclaimer: This content is for educational purposes. Accuracy is attempted but not guaranteed.

No job outcome is promised.

© Shubham Wadekar

1. Your team wants to migrate Spark ETL jobs into GCP. How would you explain why Dataflow is a good fit?

I'd start by saying Dataflow is a fully managed, serverless runner for Apache Beam that gives you consistent streaming and batch semantics, which makes it a great fit when you want to modernize Spark ETL into a unified model. In practice that means you can express the same transforms for both bounded (batch) and unbounded (streaming) data without maintaining two different code paths - fewer surprises and less operational overhead.

Second, Dataflow's autoscaling and optimized runner remove a lot of cluster management pain you have with self-managed Spark: you don't tune cluster sizes, patch nodes, or manage YARN - the service automatically adapts worker count and resources to the workload, which simplifies ops and often improves resource efficiency for variable workloads. That's especially useful if your Spark jobs currently suffer from idle clusters or complex autoscale scripts.

Third, Beam's SDKs and Dataflow provide strong windowing, event-time semantics, and built-in handling for late data and watermarks - features that are more manual or ad-hoc in many Spark codebases. If your ETL touches streaming patterns, sessionization, or needs correctness across event-time, Dataflow gives you primitives that are harder to implement reliably in plain Spark.

Fourth, Dataflow integrates tightly with GCP services (Pub/Sub, BigQuery, Bigtable, Cloud Storage, Datastore) and offers managed connectors and the Storage Write API for high-throughput, low-latency sinks. That reduces integration glue and often lowers end-to-end latency and cost compared to running Spark on ephemeral clusters that then push to these services.

Finally, I'd be pragmatic: if your Spark jobs are heavily dependent on Spark-specific libraries, complex custom JARs, or tuned Spark SQL features, a lift-and-shift to Dataproc/Spark-on-Dataproc may be faster. I'd propose a phased approach: evaluate a representative job in Dataflow/Beam, compare correctness, latency and cost versus Dataproc, then decide. That gives stakeholders data to choose the best long-term direction without disrupting production.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

2. A pipeline is scaling unpredictably. How would you analyze whether it's a control plane or worker node issue?

I'd begin by collecting and correlating logs and metrics across both layers - control plane (orchestrator, API latencies, scheduler events) and worker plane (worker CPU, memory, GC, disk I/O, network, and task retries). In an interview I'd emphasize looking for timing correlations: does the pipeline stall when the orchestrator makes calls, or do worker metrics show saturation and task slowdowns independent of scheduler events?

Next, inspect control-plane indicators: API errors, throttling responses, long scheduling delays, or spikes in orchestration latency (e.g., long task startup times, slow container provisioning). If you see rapid retries, quota errors, or long waits to acquire worker resources, that points at control-plane or quota problems rather than worker compute limits.

Then examine worker-level telemetry: high CPU, memory OOMs, excessive GC pauses, heavy shuffle/read IO, or many failed tasks on the worker side. If tasks execute slowly, fail and restart, or produce high shuffle/write throughput, it's likely a worker node resource issue. Checking task distribution helps too - if work is concentrated unevenly (hot keys) you'll see some workers overloaded while others are idle.

I'd also check external dependencies: slow downstream sinks, throttled APIs, or network congestion can look like scaling issues but are actually external bottlenecks. Correlate pipeline slowdowns with external service metrics (BigQuery quotas, Pub/Sub backlog) to see if workers are waiting on IO or being rate-limited.

Finally, draw conclusions and act: if control-plane problems, fix by raising quotas, optimizing orchestrator calls, or reducing task startup churn; if worker issues, resize clusters, tune parallelism/partitions, fix skew, or change resource configs. I'd run a controlled experiment (scale worker pools or isolate control-plane calls) to confirm the hypothesis before wide changes and document the root cause and the tuning steps so the team can prevent recurrence.

3. A client is debating between writing Beam pipelines vs using Data Fusion. How would you decide?

I'd start by mapping requirements: if the workload demands custom, complex transforms, event-time semantics, or unified batch+stream behavior, Beam (Dataflow) is usually the better technical fit because it offers expressive APIs and fine-grained control. Beam is a developer-first choice - code-centric, testable, and ideal when you need custom logic, advanced windowing, or reusable SDKs across environments.

If the priority is fast time-to-value, less engineering overhead, and a visual authoring experience with many built-in connectors, Data Fusion is attractive. It lets analysts and engineers assemble ETL quickly in a low-code way, integrates with many sinks/sources out-of-the-box, and reduces the need for hand-written code and heavyweight CI for each pipeline change - great for standard ETL workloads and operational teams that prefer GUI-driven flows.

I'd also weigh team skills and lifecycle: teams strong in Java/Python and software engineering discipline will get more long-term benefit from Beam's programmatic approach; teams with smaller engineering bandwidth or more data-ops/ETL-focused staff may prefer Data Fusion's low-code model. Consider maintainability, testing practices, and who will own the pipelines day-to-day.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Cost, scale and operational needs matter too: if you require fine-tuned performance and can optimize Beam pipelines, you may get lower operational cost at scale; if you want managed convenience, Data Fusion's UI and templates reduce ongoing ops. Also factor in hybrid requirements - if you need Spark features or Dataproc integration, Data Fusion can run on Spark runtimes which might influence the decision.

Finally, I'd recommend a short proof-of-concept for a representative job in both approaches and compare developer velocity, correctness, latency, and cost. Use that concrete evidence plus team preference to decide: pick Beam/Dataflow for complex, custom streaming logic and long-term code ownership; pick Data Fusion for rapid ETL delivery, standard connectors, and lower engineering overhead.

4. Your job requires infrastructure-free scaling. How does Dataflow's serverless design help?

I'd explain that Dataflow's value is you don't manage servers or clusters yourself. Unlike Spark on Dataproc where you size and tune clusters manually, Dataflow automatically provisions and scales worker resources based on load. If your pipeline suddenly gets a spike of data, Dataflow spins up more workers; if load drops, it scales down - all without the team having to intervene. That means you're not paying for idle compute, and you're not firefighting capacity issues when traffic surges.

Another key point is separation of control plane and worker plane. The control plane is fully managed by Google, so we don't have to patch, upgrade, or monitor the scheduler and job manager. Worker nodes are ephemeral and stateless from our perspective - Dataflow spins them up, replaces them on failure, and handles retries automatically. This design makes jobs resilient to hardware failures and removes operational burden.

For our job, which needs "infrastructure-free scaling," this means we can focus entirely on writing transforms in Beam and not worry about provisioning. We just submit the job and Dataflow takes care of where and how it runs. That also ensures consistent SLAs across batch and streaming - the same code runs on an environment that adapts to the workload, which is hard to achieve with self-managed clusters.

Finally, I'd highlight cost and reliability: because scaling is elastic, we avoid overprovisioning for peak, and because it's serverless, Google ensures high availability and autoscaling out-of-the-box. That combination - no servers to manage, no manual autoscaling, and strong resiliency - is exactly what's meant by "infrastructure-free scaling."

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

5. You must convince management that using Apache Beam with Dataflow provides future flexibility. How would you explain?

I'd frame it in terms of future-proofing and avoiding lock-in. Apache Beam is an open-source programming model that's portable across multiple runners - Dataflow, Spark, Flink, and others. If we build our pipelines in Beam, we're not tied to one runtime forever. Today we get the benefits of Dataflow's serverless scaling, but if in the future cost, compliance, or hybrid-cloud strategy changes, we can move the same Beam pipelines to another runner with minimal changes.

Another advantage is unifying batch and streaming. Beam lets us write one pipeline definition that can process both bounded (historical) and unbounded (real-time) data. That means we don't have to maintain separate codebases for ETL vs streaming, which reduces maintenance cost and gives flexibility as business needs shift from daily batch to near real-time analytics.

I'd also point out ecosystem and skills: Beam has a growing open-source community, SDKs in Java and Python, and many connectors. Engineers we hire will already know or be able to learn Beam, and the pipelines won't become obsolete if we decide to leave Dataflow. This gives management confidence that we're investing in a long-lived technology, not a niche product.

Finally, Beam plus Dataflow gives us both worlds: Google handles operational scaling, tuning, and reliability, while Beam keeps the programming model open and portable. That's a safe investment - we get immediate benefits now and the freedom to adapt in the future without a costly rewrite. For management, that's both risk mitigation and long-term flexibility.

6. You need to process 10TB of historical logs once and then stream new events daily. How would you design this hybrid pipeline?

Start with a clear separation of concerns: treat the 10TB historical load as a one-time bulk job and new events as an ongoing streaming job. For the history, use a bulk-efficient path - stage files in GCS (Parquet/Avro preferred), then use BigQuery load jobs or a Dataflow batch job to ingest into partitioned staging tables. Bulk load is cheaper, faster, and more resilient for large one-time imports than trying to stream 10TB through a streaming pipeline.

Next, normalize schema and create a canonical staging area that both history and streaming feeds write into. Ensure the historical load writes into the same partition/column conventions as the streaming path (same field names, time partition column, audit columns like source_batch_id and ingestion_ts). This makes downstream SQL and materialized views consistent and avoids special-casing consumers.

For the ongoing stream, use Pub/Sub → Dataflow (Beam) → BigQuery (or storage-write API) with event envelope including event_id, event_ts, and schema_version. Apply the same validation/cleaning logic as the historical batch (ideally reuse transform code or SQL), and write to the same staging tables or directly to production tables with idempotent MERGE semantics. Keep raw events archived in GCS for replayability and auditing.

Make everything idempotent and re-playable: use manifests and checksums for the historical load, and include insertIds or primary keys for streaming so replaying events or re-running the bulk job won't create duplicates. Provide a reconciliation step (row counts, checksums per partition) after the bulk job and regularly for streaming to prove parity and catch drift early.

Finally, operationalize and monitor: schedule the bulk job during a low-cost window, verify partition-level validation, then cutover streaming ingestion. Add alerts for ingestion lag, DLQ growth, and partition load mismatches. Document runbooks for reprocessing historical partitions or replays from the raw GCS archive so you can recover or iterate on transforms without losing data integrity.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

7. A business requires daily reports at 7 AM from yesterday's data. Would you use batch or streaming? Why?

I'd choose batch for this use case. A daily 7 AM report has a fixed latency requirement (data up to end-of-day yesterday), so a scheduled batch that runs in a predictable window is simpler, cheaper, and easier to reason about than maintaining a full streaming pipeline for data that only needs to be fresh once per day.

Batch also lets you use efficient bulk load mechanisms and heavy SQL-based transformations in BigQuery without the per-record overhead or continuous-cost profile of streaming. You can run a nightly job that ingests and compacts data into partitioned tables, runs dedupe/merge logic, and computes aggregates for the report - all in a cost-effective manner with built-in validation before the 7 AM SLA.

Operationally it's easier to guarantee correctness: schedule the pipeline to run after source systems finish producing (or after an agreed buffer window), perform full-row validation and reconciliation, and fail loudly if counts don't match. That deterministic behavior is often preferred for business reporting where accuracy matters more than ultra-low latency.

That said, if parts of the report need near-real-time metrics or an early-warning dashboard throughout the night, you can hybridize: maintain a light streaming pipeline for near-real-time alerts and a heavy batch that runs final, authoritative computations for the 7 AM report. This gives both immediacy and a single source of truth for the daily report.

Finally, keep practical details in mind: partition the BigQuery tables by date, precompute summary tables or materialized views for the 7 AM query patterns, and schedule the batch with a robust orchestrator (Cloud Composer / Cloud Scheduler) with retries and notifications so stakeholders always get a timely, validated report.

8. A pipeline must calculate rolling metrics every minute. Which mode would you choose?

For one-minute rolling metrics I'd choose streaming mode. Windowed streaming (event-time windows, sliding windows) is the natural fit for frequent rolling aggregates because it provides low-latency updates, accurate event-time semantics, and built-in handling for late-arriving data and watermarking - all things batch or micro-batch struggle to do cleanly at per-minute granularity.

Implement this with Pub/Sub → Dataflow (Beam): key by the entity (user/session/device), apply a sliding or tumbling window (for example a 1-minute sliding window updated every 10s or a 1-minute tumbling window depending on required overlap), and use stateful/windowed aggregations to emit metric updates. Use event-time processing and watermarks so late events are included correctly and you don't overcount or miss updates.

Make the output idempotent and consumer-friendly: include window start/end, event counts, and watermark metadata; write to a partitioned BigQuery table, a timeseries DB, or push updates to a low-latency store or cached layer (like Redis or Bigtable) if dashboards require super-fast read performance. If cost is a concern, consider emitting only deltas or using micro-batching within Dataflow to reduce per-message overhead while keeping minute-level latency.

If strict per-minute windows are relaxed a bit (e.g., acceptable 30–60s latency and you want lower cost), a micro-batch approach (small batches every 30s–60s) can be used, but it sacrifices precise event-time semantics and late data handling. For correct, production-grade rolling metrics at one-minute cadence, streaming with proper windowing is the safer, more accurate choice.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Finally, instrument for operations: monitor processing latency, watermark progress, window completeness, and DLQ rates. Tune parallelism and state TTLs to avoid memory blow-ups, and add a replayable raw archive so you can re-run windows for historical corrections if needed.

9. Your streaming pipeline has high cost. How would you decide if switching to batch is viable?

I'd start by quantifying the true latency requirement: ask which consumers need data in real time and what "real time" means in seconds or minutes. If stakeholders can tolerate minutes-to-hours latency, batch or micro-batch becomes a candidate; if they need sub-second visibility (fraud, alerts), streaming may be non-negotiable. Put those SLAs in clear, measurable terms and map them to business impact before changing architecture.

Next, measure cost drivers and profile the streaming job: CPU/worker hours, number of workers always-on, API call frequency, BigQuery streaming insert costs, Pub/Sub throughput, and DLQ/retry overhead. Compare that to an estimated batch cost model (scheduled Dataflow/Data Fusion runs or BigQuery load jobs) using realistic batch sizes and frequencies. This concrete cost-per-day or cost-per-month comparison makes the tradeoff tangible.

Then, evaluate data semantics and correctness needs: streaming provides event-time windowing, watermarking and low-latency dedupe. If accuracy depends on event-time semantics (late arrivals, sessionization), determine whether batch windows + watermark-tolerant logic or a hybrid approach (micro-batches with replayability) can preserve correctness. If exact ordering or sub-second consistency is required, batch may not be acceptable.

Also consider operational complexity and TCO: streaming often requires always-on jobs, more complex monitoring, and DLQ handling - those are hidden costs. Batch simplifies operations and can leverage cheaper bulk-load paths (BigQuery load jobs, partitioned files). Account for developer time, monitoring overhead, and risk of data staleness when comparing total cost of ownership, not just raw cloud charges.

Finally, prototype and validate: run a short POC that implements the batch alternative (e.g., 1–5 minute micro-batches and a nightly full batch) and compare latency, cost, and correctness on representative traffic. Use that evidence plus stakeholder tolerance for latency to decide: choose batch when it meets SLAs and materially reduces cost; choose hybrid when some consumers need near-real-time while others are fine with delayed, authoritative data.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

10. Your team wants to unify historical + real-time datasets in one pipeline. How would you achieve this?

Design a canonical ingestion layer that both paths write into: store raw events from historical loads (bulk GCS files) and streaming events (Pub/Sub) into the same staging schema in BigQuery or as partitioned Parquet files in GCS. Ensure both flows produce the same envelope (unique id, event_ts, schema_version, source_tag, batch_id) so downstream logic sees a single, consistent input format regardless of origin.

Reuse transformation logic by implementing shared, idempotent transforms. Prefer writing the core cleaning/enrichment logic once (as Beam transforms, SQL in BigQuery, or a reusable Data Fusion pipeline) and call it from both the bulk job and the streaming job. That prevents divergence and ensures historical reprocessing uses the same business rules as real-time ingestion.

Handle ordering, deduplication and late-arriving data consistently: include a reliable primary key and commit timestamp in each record, and apply the same MERGE/upsert strategy into the final production tables. If you need event-time correctness, use Watermarks in streaming and apply similar window-aware aggregation logic in the batch refreshes or backfills so aggregates remain consistent across modes.

Architect for replayability and reconciliation: keep raw archives for both historical and streaming inputs (GCS or a changelog table) and maintain a manifest of processed ranges or offsets. Build reconciliation jobs that compare counts and checksums per partition between raw and final tables so you can detect and fix gaps or duplicates and re-run transforms on historical slices as needed.

Operationalize and document the unified approach: parameterize pipelines (source_type, partition_date, batch_id), add CI tests that run transforms against both batch and streaming sample inputs, and monitor parity metrics (lag, duplicate rate, partition completeness). This gives teams a single logical pipeline for consumers while keeping the operational differences between bulk loads and streaming manageable and auditable.

11. You must transform records, filter errors, and log metrics. Which Beam constructs would you use?

I'd use a ParDo with a custom DoFn as the primary workhorse - it's the simplest, most flexible place to put per-record transformations (parsing, type-casting, enrichment). Inside the DoFn I'll perform the transform logic and emit the clean record to the main output only when it passes validation. ParDo lets you encapsulate complex, multi-step logic in one place and keeps the pipeline readable.

For filtering errors I'd use *tagged outputs* (side outputs) from the same DoFn - emit malformed or business-rule-failing records to a dead-letter PCollection while letting the main, valid records continue downstream. That makes the flow explicit and gives you a separate path for retries, inspection, or storage to GCS/BigQuery without failing the whole job.

For operational visibility use Beam's Metrics API inside the DoFn - counters for total/valid/invalid records, distributions for latency or field sizes, and gauges where supported. Increment counters when you encounter specific error categories (parse error, missing key, invalid timestamp) so you can alert on spikes and drill into common failure modes without sampling raw data.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

If you're streaming and need time-awareness, wrap the ParDo with appropriate windowing and watermark-aware triggers so late records are handled consistently; combine the ParDo with WithTimestamps before transforms if the timestamp is embedded in the payload. Also consider using Reshuffle or Window.into only where needed to avoid unnecessary global synchronization.

Finally, route the dead-letter PCollection to durable storage (GCS or a DLQ topic) and add a small replayable remediation step that reads the DLQ, applies fixes, and republishes. That keeps the main pipeline running smoothly while preserving failed records for forensic and reprocessing work - and with metrics attached you can monitor both success and failure rates in real-time.

12. You're asked to aggregate purchase data by customer. Which Beam transform would you pick?

For per-customer aggregation the canonical pattern is KeyBy (or Map to (customerId, record)) followed by CombinePerKey (or GroupByKey + a combiner) - CombinePerKey is preferred when you can express the aggregation as a combiner (sum, count, min/max, or a custom combiner that maintains intermediate state). CombinePerKey is efficient because it does local pre-aggregation before shuffling, which reduces network IO on large datasets.

If you need sliding or tumbling time windows (e.g., 1-hour rolling spend), apply Window.into(...) before the CombinePerKey so the aggregation is computed per-window. Use event-time windows with appropriate watermarking and late-data policies so the aggregates are correct even with delayed events. For streaming you'll also pick triggers (default or custom) based on how early/late you can emit partial vs final results.

For more complex aggregations (top-N items per customer, approximate distinct counts, histograms) write a custom CombineFn that maintains the necessary accumulator shape; this keeps computation incremental and performant across the distributed runners. If aggregates are complex and stateful per key, consider Stateful DoFn keyed by customer to manage finer control, but prefer CombinePerKey for most simple-to-moderate use cases.

If absolute ordering or exactly-once semantics matter for financial totals, pair CombinePerKey with idempotent sink semantics (use BigQuery MERGE with a transaction key or Storage Write API with dedupe insertId) so retries or replays don't overcount. Also emit audit metrics (counts, watermark lateness, bytes) so downstream users can trust freshness and correctness.

Finally, test the aggregation on representative windows and scale - validate with small unit tests and an end-to-end run using sample traffic to ensure the combiner behaves correctly under parallelism, late data, and reprocessing scenarios.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

13. A lookup table is needed inside a transform. How would you implement this?

If the lookup table is small (fits comfortably in memory), use a *side input* (e.g., AsDict / AsSingleton depending on shape) and pass it into your ParDo/DoFn. Side inputs are effectively broadcast to workers, very fast at lookup time, and straightforward to use for reference data like country codes, SKU-to-category maps, or small dimension tables.

For moderate-sized or frequently-changing reference data, use a cached side input pattern: read the reference data into a side input and refresh it periodically (e.g., re-run the pipeline or use small windowed reloads). If you need versioning, include a schema_version or valid_from field so transforms can choose the right mapping. Be careful about update latency - side inputs are snapshot-in-time for the bundle/job that sees them.

If the lookup is large or cannot be broadcast, do a distributed join: convert the lookup into a PCollection and perform a CoGroupByKey (i.e., key both the main records and the lookup by the join key and then join). This pattern scales because it shuffles both sides by key, but you should repartition and possibly pre-aggregate the lookup side to reduce size. For very large lookups that are queried heavily, consider materializing them in an external low-latency store (Bigtable, Memorystore/Redis) and perform async I/O from DoFn - use asynchronous DoFns to batch and parallelize network calls to avoid high latency.

When using external lookups, design for retries, throttling, and caching: batch requests where possible, add a local LRU cache per worker, and backoff on 429/5xx responses. For high-throughput pipelines prefer an external store that supports bulk reads (e.g., pre-fetch ranges into side inputs) or use an on-cluster cache populated at job startup.

Finally, always make the lookup behavior deterministic and testable: include unit tests with mocked side inputs or a small in-memory lookup, add monitoring for cache hit/miss rates, and document how/when the lookup data is refreshed so consumers understand eventual consistency behavior.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

14. You want to split output into multiple categories. How would you achieve this?

I'd first decide the routing rules (field-based, threshold-based, ML model, etc.) and make them explicit so the pipeline is deterministic. In code (Beam) I'd implement a ParDo/DoFn that evaluates each record and emits to different *tagged outputs* (side outputs) or returns a tuple key that downstream Partition/GroupByKey can use. In Data Fusion I'd use the Router/Filter transforms or multiple output ports with clear predicates so the visual pipeline shows the routing logic.

Next, I'd keep the routing logic simple and centralized so it's easy to change and test - one place owns the decision and emits the category meta. That function should also attach metadata (category, routing-reason, pipeline-version) to each record so downstream consumers know why a record landed in a bucket and can handle versioning or audits.

For sinks, pick the right pattern: write each category to a separate sink (BigQuery tables, GCS prefixes, Pub/Sub topics) or use dynamic destinations (BigQuery dynamic table spec, partitioned GCS paths) if categories are many or created at runtime. Ensure the sinks are idempotent (MERGE, insertId, atomic rename patterns) so a replay doesn't double-write across categories.

Operationally, emit metrics per category (counts, error rates) and sample records for each path so you can validate routing correctness. Add a dead-letter branch for records that don't match any category or fail evaluation so nothing is lost silently.

Finally, test thoroughly: unit-test the routing logic with edge cases, run an end-to-end smoke test that exercises each category, and document the contract (category names, predicates, expected cardinality). That makes the routing maintainable, observable, and safe to evolve.

15. A requirement asks for keeping session state per user. How would you design this?

I'd key the stream by user_id (or session_id) so state and timers are scoped per key, then use a stateful DoFn with per-key StateSpec and TimerSpec (Beam/Dataflow) to hold session aggregates - last activity timestamp, event buffer, counters - and a timer to close/invalidate sessions after inactivity. This gives precise control: you update state on each event and use the timer to emit session summaries when a session ends.

Use event-time windowing (session windows) when possible because it naturally groups events into sessions by activity gaps and works with watermarks/late data. If you need richer semantics (multi-step state, cross-event matching), implement stateful DoFn so you can maintain arbitrary per-session structures and emit partial/final results on timer firings.

For scale and resilience, rely on a managed runner (Dataflow) that shards keys and manages state backend; keep state compact and set TTLs so you don't retain forever. If sessions must survive worker restarts or be queried externally, persist checkpoints or canonical session snapshots into a durable store (Bigtable or Redis) for long-term access and cross-process querying.

Make the design idempotent and tolerant of duplicates: include event ids and last-processed offsets in state so replayed events don't corrupt aggregates. Also codify late-arrival handling - either allow updates to a recently-emitted session or buffer and re-emit corrected session summaries downstream with a version or correction flag.

Finally, add monitoring: per-key state size distribution, timer backlog, session-duration histograms, and alerts if state growth or timer lag spikes. Unit-test the state logic with synthetic time-advanced tests and include runbooks for replaying or repairing corrupted sessions. This keeps session handling correct, scalable, and operable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

16. You need hourly aggregations of streaming events. Which windowing strategy would you use?

I'd use event-time, fixed (tumbling) hourly windows aligned to the clock (e.g., window start at 00:00, 01:00, etc.). Event-time windows ensure aggregates reflect when events actually happened, not when they were processed, which is critical for correctness when sources have variable delays. Fixed hourly windows are simple to reason about and map directly to reporting partitions.

I'd set the pipeline's watermarking so the system knows when a window is "mostly complete." The watermark should be tuned based on observed source delays (for example, watermark = max event time seen − X minutes) so you don't wait unnecessarily long for late events but also avoid emitting too-early, incorrect results. Picking a realistic watermark offset is one of the most important operational knobs.

I'd combine the fixed window with an appropriate trigger and allowed-lateness (see next questions) so we can emit early, updated, and final results as data arrives. For aggregation I'd use CombinePerKey (or an equivalent combiner) inside the window so pre-aggregation happens before shuffle and scales well for large keyspaces.

Make sure the output sink and downstream consumers can handle incremental updates (emit full-window aggregates or deltas, include window start/end, version or watermark metadata). Also design the write semantics to be idempotent - e.g., use a MERGE by key+window when writing to BigQuery - so retries or re-emissions don't corrupt counts.

Finally, monitor window completeness: expose metrics for on-time vs late arrivals, fraction of late data, and window emission latency. That helps you iterate on watermark/allowed-lateness settings and trade accuracy vs latency in production.

17. A pipeline must produce partial results before all data arrives. How would you configure triggers?

I'd use the watermark-based end-of-window trigger for the "final" result and add early triggers to emit partial results before the watermark. Concretely, use AfterWatermark.pastEndOfWindow() as the main trigger and add .withEarlyFirings(AfterProcessingTime.pastFirstElement().plusDelayOf(Duration.standardSeconds(...))) (or an AfterPane.elementCount trigger) to emit early panes at a regular cadence or after a certain number of elements.

Choose an early trigger based on the latency/accuracy tradeoff: use time-based early firings (e.g., every 30s or 1m) for streaming dashboards that need continuous updates; use element-count early firings if you want updates after meaningful volume arrives. Combine that with accumulation mode: if you want each early firing to represent the aggregate up to that moment, use ACCUMULATING mode; if you want each pane to be independent, use DISCARDING - ACCUMULATING is typical for incremental rolling metrics.

Also configure late firings for out-of-order events: set withLateFirings(...) or rely on allowed-lateness so when late events arrive within the allowed window you can update the previously emitted aggregate. Make sure the sink can accept updates (e.g., MERGE into BigQuery or write delta rows with window+key so downstream clients can reconcile).

Be explicit about how partial results are labeled: include window start/end, a pane-timestamp or watermark snapshot, and a flag indicating whether this pane is "early", "on-time final", or "late update." That metadata prevents consumers from misinterpreting transient, partial numbers as final figures.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Finally, test the trigger behavior with synthetic late and early event patterns to verify early/downstream update behavior, and monitor the frequency of late updates - if late updates are frequent, consider increasing allowed-lateness or changing the watermark strategy.

18. Your events arrive late by 10 minutes. How would you adjust windows?

I'd set allowedLateness to at least 10 minutes (and usually a bit more as a safety margin) so windows remain open to accept late events arriving within that window. In Beam/Dataflow this means you'd configure the window's allowed lateness and ensure the runner retains state for that duration so late elements can be incorporated and late firings emitted.

Tune the watermark lag to reflect source characteristics: the watermark should be behind event time by an amount that reasonably covers typical latency (e.g., watermark = maxEventTime − 5–8 minutes) while leaving allowed-lateness to catch the tail (the 10-minute arrivals). The watermark and allowed-lateness work together - watermark controls when the "on-time" firing happens, allowed-lateness controls how long you accept and handle late arrivals after that.

Because late arrivals will cause updates after initial emits, design your sink writes to be updatable/idempotent: use MERGE by key+window or write correction/delta records with versioning so downstream systems can apply late corrections. Also consider using ACCUMULATING mode so each late firing adds to the prior aggregates rather than discarding previous panes.

Monitor the proportion and timing of late arrivals. If many events consistently arrive ~10 minutes late, you may want to shift your watermark strategy (e.g., increase watermark lag) so the "final" on-time emission waits longer, reducing the number of late updates - at the cost of higher latency for on-time results. Tune this tradeoff with stakeholders: fewer late corrections vs faster initial visibility.

Finally, test the full lifecycle: simulate a mix of on-time and 10+ minute late events, verify your triggers/fireings behave as intended, ensure state retention doesn't blow up memory (set TTLs as appropriate), and document the expected behavior so downstream consumers can interpret early vs final aggregates correctly.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

19. A client wants results to be updated instead of duplicated. Which accumulation mode would you choose?

I'd choose accumulating mode (ACCUMULATING_FIRED_PANES) so each firing for a given window represents the aggregate of everything seen so far rather than emitting independent, disjoint panes. That way downstream consumers can update the previously shown value (replace or merge) instead of treating each pane as a separate, additive row.

Practically, picking accumulating mode must be paired with idempotent/upsertable sinks - e.g., write results keyed by window_start + aggregation_key and use a MERGE/upsert into BigQuery or a storage-write method that can replace the prior pane. Without an upsert-capable sink, accumulating output will still appear duplicated to consumers.

You also need to include clear metadata with each output (window start/end, pane timestamp, a version or watermark snapshot) so consumers can decide whether to replace or ignore an incoming pane. For example a dashboard can overwrite the cell for customer X / window Y when it gets a later accumulating pane with a higher pane timestamp.

Handle late-arriving data explicitly: allow a reasonable allowed-lateness so late events update the accumulated window, and design retention/state TTLs to hold window state until late arrivals are unlikely. Monitor the frequency of late updates - if they're common you might tune watermark or allowed-lateness to balance freshness vs correctness.

Finally, add tests and runbook: unit tests that simulate early/late firings and verify the sink gets correct upserts, plus runbook instructions for how to reconcile or re-run if a late correction must be backfilled. That keeps the "update not duplicate" behavior reliable and auditable.

20. You see data missing in windows due to watermark misalignment. How would you fix it?

First I'd verify the root cause: check whether the pipeline is assigning event timestamps correctly (use WithTimestamps on the real event-time field) and inspect watermark progress metrics - if the watermark advances past the event times too quickly, late events will be dropped or assigned to the wrong window. Often the problem is wrong timestamp extraction or relying on processing-time instead of event-time.

If timestamps are correct but the watermark is too aggressive, increase the watermark lag or implement a more conservative watermark estimator. In Beam you can adjust how you compute watermarks (or rely on the runner's default but tune its parameters) so the watermark advances only after a plausible delay that covers typical source lateness and network jitter.

Also adjust allowed-lateness for the windows so you accept events that arrive after the watermark by your expected lateness (e.g., if events commonly arrive 10 minutes late, set allowed-lateness ≥ 10m). That will keep windows open to accept and incorporate late arrivals instead of treating them as missing.

If skew or hot keys cause localized delays, add preprocessing to smooth skew (salting, pre-aggregation) or change partitioning so no single slow shard delays watermark progression for unrelated keys. And if the source (e.g., Pub/Sub) doesn't provide reliable event-time progression, consider injecting a safe event-time heuristic (like using a producer timestamp plus a bounded buffer) before the windowing stage.

Finally, validate changes with controlled tests: replay synthetic late/on-time mixes, monitor watermark and late-element counts, and verify that previously-missing records now land in the expected windows. Document the chosen watermark and allowed-lateness policy so future changes don't reintroduce the misalignment.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

21. IoT events sometimes arrive an hour late. How would you design your pipeline to handle this?

Start by treating event-time as the source of truth and make sure every message carries a reliable event timestamp; assign that timestamp in the pipeline (WithTimestamps) so all downstream windowing uses event-time, not processing-time. That gives you correct grouping even when messages arrive late.
Use a windowing + allowed-lateness strategy that explicitly covers the observed tail - for hour-late events set allowedLateness >= 1 hour (add a safety margin) so windows remain open to incorporate late arrivals rather than dropping them. Combine this with appropriate triggers: emit early partial results for low-latency needs and rely on the final firing after allowed-lateness for authoritative values.
Because keeping windows open for an hour increases state, limit per-key state size with TTLs and compact accumulators; if sessions or keys are extremely high-cardinality, shard state or offload long-term session snapshots to a durable store (Bigtable) and only keep short-term incremental state in the runner. Tune state retention to balance memory and correctness.
Persist every raw event immediately to an immutable archive (GCS or BigQuery raw table) as the first step. That archive becomes your source of truth for replays/backfills and lets you re-run window computations for any late-arrival window deterministically if you need to fix historical aggregates.
Make sink writes idempotent and update-friendly - write windowed results keyed by window_start + dimension and use MERGE/upsert semantics (or insertId with dedupe) so late updates replace prior partial values instead of creating duplicates. Finally, monitor late-event rates, watermark lag, and window-update frequencies and include runbooks for backfill and reprocessing when SLA violations occur.

22. A watermark is advancing too quickly, dropping late data. What would you do?

First, verify you're assigning timestamps from the correct event field and not accidentally using processing time; wrong WithTimestamps is a common root cause. If timestamps are correct, the watermark estimator is likely too aggressive - adjust the watermark lag to be more conservative so it advances more slowly and allows the observed lateness window.
Increase allowedLateness for affected windows so late events within your expected tail are still accepted. Pair that with a conservative watermark and consider switching some windows to session or sliding windows if the lateness pattern is bursty or tied to user behavior rather than clock boundaries.
If watermark advancement is driven by a subset of sources pushing very late high-event-time values (skew), isolate or pre-process those sources: tag them, route through a buffering stage, or inject a bounded hold so their outliers don't move the global watermark. Alternatively, compute watermarks per-partition/key if your runner and design support localized watermarks.
Inspect and tune runner-specific watermark settings (Dataflow has watermarking configuration and heuristics) and test with replays that simulate late arrival patterns to confirm behavior. Add metrics for watermark age, late-element counts, and windows closed early so you can detect regressions.
Finally, make downstream consumers tolerant of updates: emit pane metadata (pane type, watermark snapshot) and ensure sinks support updates/merges so when late data does arrive it can update prior results rather than being permanently lost.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

23. A downstream system requires complete accuracy, but your pipeline is discarding late events. How would you redesign?

Accept that when absolute accuracy is required you must move from optimistic streaming-only finalization to an authoritative, replayable model: persist every raw event to an immutable changelog (GCS or a BigQuery raw table) as soon as it arrives, and treat streaming computations as speculative/near-real-time views while the changelog is the single source of truth for final calculations.
Adopt a two-phase output model: emit provisional/early results for low-latency consumers but mark them clearly as provisional; produce authoritative final aggregates only after a deterministic window (watermark + allowed-lateness) or after an explicit reconciliation job that reprocesses the changelog for the affected time ranges. The reconciliation job writes final, versioned results into the downstream system using idempotent MERGE/upsert semantics.
Make all writes idempotent and versioned: key final rows by entity + window + version and use MERGE so replays or late-updates overwrite previous values. Keep a last_update_ts and source_batch_id to allow downstream systems to detect and accept newer authoritative updates. For transaction-critical flows consider adding a commit-state column (pending/committed) so consumers only treat committed rows as authoritative.
If the downstream system cannot accept updates easily, add a reconciliation layer that emits compensating transactions (reversals/deltas) rather than overwrites, or use an intermediate staging table where finalization happens and the downstream pulls only finalized snapshots.
Provide a deterministic replay API or tool that can re-run slices of the changelog to repair any gaps.
Finally, instrument reconciliation: build automated parity checks (row counts, key-level checksums) between changelog and downstream state, alert on mismatches, and run periodic backfills. Document SLAs for provisional vs final data and include runbooks so operators can reprocess, audit, and certify that downstream state is finally accurate.

24. Your pipeline is overloaded with late events. How would you balance latency vs accuracy?

First I'd quantify the problem: measure how late events are, their volume, and their business impact - which consumers need low latency and which need accurate, final results. With concrete numbers I can make an explicit tradeoff instead of guessing: e.g., 80% of events arrive within 2 minutes but 20% arrive up to an hour late and they affect settlement totals vs dashboard KPIs differently.

Second, split consumers by SLA and provide two classes of outputs: provisional/near-real-time views for low-latency consumers and authoritative/final views for accuracy-sensitive consumers. Emit early, best-effort aggregates (with a "provisional" flag and pane watermark) while scheduling a deterministic reconciliation pass after a configured allowed-lateness to produce final values that overwrite or MERGE into the authoritative table.

Third, tune windowing/watermark parameters pragmatically: increase allowed-lateness or shift the watermark to reduce late updates if accuracy is prioritized, or shorten allowed-lateness and accept more late corrections if latency is more important. Use early triggers with accumulating mode so partial results are useful, then rely on the final firing (after allowed lateness) for truth - tune the timings based on the measured lateness distribution.

Fourth, reduce operational pressure by pre-aggregating or buffering problem sources: if a small subset of sources causes most late events, isolate them, buffer their events a bit longer, or apply separate windowing for them so they don't delay the global watermark. Also consider micro-batching (1–5 minute windows) as a middle ground - lower cost and simpler semantics than per-event streaming, but much fresher than daily batch.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Finally, make the behavior transparent and automatable: document SLAs for provisional vs final data, expose freshness/late-arrival metrics on dashboards, and automate reconciliation (replay from raw archive/GCS) when finalization finds discrepancies. That way stakeholders know when to trust provisional numbers and you have an auditable path to correct final results.

25. You need to capture all late events separately for audits. How would you implement this?

I'd treat late events as first-class citizens and capture them at the ingest boundary into an immutable audit store immediately - write every raw event (with producer timestamp, ingestion timestamp, topic/partition metadata, and pipeline version) to a GCS archive or a BigQuery raw changelog table before any filtering or transformation. That guarantees you have the original payload for forensic analysis regardless of pipeline behavior.

Next, in the processing pipeline detect late arrivals by comparing event timestamp to current watermark and route those late records to a dedicated "late-events" branch or topic. Persist these late events (and the reason/late_delta) into a durable audit table or partitioned GCS path, including the window they pertain to and any pane metadata so auditors can reconstruct exactly when and why they were late.

Third, enrich the audit records with provenance metadata - which pipeline run saw it, transform version, attempt count, and any intermediate fixes applied - so reprocessing or manual remediation has full context. Keep the late-events store read-optimized (partitioned by event date and ingestion date) so auditors can query slices efficiently without scanning your raw archive.

Fourth, provide replay and reconciliation tools: a small utility or orchestration job that can read the late-events archive, apply fixes or updated parsing rules, and replay them into the canonical pipeline or a special reprocess topic with idempotent semantics. Include test harnesses so auditors or engineers can validate corrections before committing them to production tables.

Finally, operationalize access and governance: secure the audit store with strict IAM, document retention policies to meet compliance, surface metrics (late-event counts, age distribution) to trigger alerts when late rates spike, and include runbooks for how to investigate, remediate, and certify corrected results. This gives auditors both the raw evidence and the operational controls they need.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

26. You must clean, normalize, and enrich records before writing to BigQuery. Which Beam transforms would you use?

I'd start by assigning proper event timestamps if they aren't already present using WithTimestamps, so downstream windowing and late-data handling work correctly. Getting timestamps right early avoids subtle bugs later and makes enrichment logic (like time-based joins) deterministic.

Next, implement a ParDo (DoFn) as the main cleaning/normalization stage: parse raw payloads, canonicalize strings (trim/case/normalize), validate required fields, and do safe casts with fallbacks. Inside the DoFn I'd also emit malformed records to a tagged side output (dead-letter) so bad rows don't fail the whole job but are preserved for inspection and replay.

For enrichment I'd use side inputs for small/medium lookup tables (AsDict or AsSingleton) so the transform can do very fast in-memory lookups. If the reference data is large, I'd model it as a PCollection and perform a scalable join via CoGroupByKey or use an async external lookup (e.g., Bigtable/Redis) from an asynchronous DoFn to avoid blocking workers. Cache and refresh side inputs periodically if the reference data changes.

Before writing to BigQuery, apply a MapElements or another ParDo to shape records to the exact BigQuery schema, add audit fields (ingestion_ts, pipeline_version, source_file), and validate final types. For writes, use BigQueryIO.write() (Storage Write API or load jobs depending on volume and latency) with idempotent keys or partitioned tables - prefer load jobs for bulk batches and Storage Write API for low-latency streaming, always ensuring dedupe/merge semantics are supported.

Finally, instrument with Beam Metrics inside transforms (counters for cleaned/invalid/enriched counts, distributions for latencies), route dead-letter/output samples to GCS or a DLQ, and include downstream validation (row-count and checksum comparison) after the BigQuery load so you can detect mismatches quickly and reprocess if needed.

27. You're asked to join a large dataset with a smaller reference dataset. How would you design this?

I'd first measure the small dataset's size and update frequency. If it's small enough to fit comfortably in worker memory (tens of MBs to a few hundred MBs depending on cluster), I'd broadcast it as a side input (AsDict) and perform a fast in-memory lookup inside a ParDo. That gives the lowest-latency and simplest implementation with minimal shuffle.

If the reference data is larger or frequently changing beyond what a single broadcast can handle, I'd convert it to a keyed PCollection and use CoGroupByKey (i.e., a distributed join) so both sides are shuffled by key and joined at scale. To reduce shuffle cost, pre-aggregate or prune the reference side where possible and ensure keys are normalized to avoid skew.

When skew is an issue (very hot keys), I'd apply salting on the large side and replicate the small side appropriately, or use a two-phase approach: pre-aggregate or cache the hot-key results, then join the remainder normally. For extremely high-throughput needs, consider writing the reference table to a low-latency store (Bigtable/Redis) and perform async lookups from workers-this avoids heavy shuffles at the cost of external calls and extra operational complexity.

Also consider pushing the join down to the storage/warehouse if data is resident there (e.g., perform the join in BigQuery via a staging table), especially for one-off or bulk jobs-often cheaper and simpler than a distributed join in the pipeline. Whatever approach you choose, make it idempotent and test it with representative data to validate performance and correctness.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Finally, include monitoring for join completeness (counts, null-join rate), handle missing references explicitly (put them in an extensions/audit column or DLQ), and document the join choice and fallback so future engineers know why side-input/broadcast vs CoGroupByKey was chosen.

28. You need to route events to multiple sinks based on event type. How would you partition them?

I'd centralize the routing rules in one deterministic function and implement them in a ParDo that either uses tagged outputs (side outputs) or returns a typed routing key. Tagged outputs are great for Beam/Dataflow: the DoFn evaluates event type and emits the record to the appropriate PCollection for each sink (BigQuery table, Pub/Sub topic, GCS path), and one record can be emitted to multiple tags if needed.

If using BigQuery or other dynamic destinations, I'd prefer dynamic destinations APIs (e.g., BigQueryIO.write().to(Function<...>)) so the write step itself decides the table or partition path based on event metadata - this keeps routing logic close to the sink and reduces intermediate shuffles. For file sinks, write to partitioned prefixes like gs://bucket/events/type=click/date=.../ to make discovery easy.

For high-cardinality or runtime-created event types, use a generic partitioning scheme: map event type → topic/table name via a config table (side input) so you don't hardcode sinks. If throughput is high, buffer/pool writes and batch per-destination to reduce RPC overhead. Ensure each sink's write is idempotent (insertId for Pub/Sub, MERGE for BigQuery, atomic rename for GCS) to safely handle retries.

Operationally, emit per-sink metrics (counts, error rates), add a default/unknown sink for unexpected types (and surface those events to DLQ), and include schema/version in each routed message so consumers know how to parse them. Also document routing rules and provide a small replay tool to republish messages to the correct sink after fixes.

Finally, test end-to-end routing with representative volumes and the full set of sinks, validate that latency and partitioning meet consumer SLAs, and make the routing function configurable (toggle rules, add new sinks) without code changes when possible.

29. A business wants per-key aggregations on billions of records. How would you implement this efficiently?

I'd start by choosing a scalable, distributed aggregation primitive - in Beam that's KeyBy + CombinePerKey (or a well-designed CombineFn). CombinePerKey lets the runner do local pre-aggregation before the global shuffle, which drastically reduces network I/O for huge volumes. Design the combiner to be associative and commutative so it can safely reduce partial results on workers before the full shuffle.

Next, tune partitioning and parallelism so the work is evenly distributed. Don't let a single global GroupByKey create a bottleneck - pick good keys (or shard/augment keys) and set an appropriate number of shuffle partitions. If you expect skew (hot keys), detect heavy hitters and handle them separately - either replicate the small side, salt the hot key (split into multiple logical keys and un-salt after aggregation), or pre-aggregate hot-key streams in a heavyweight path to avoid overwhelming a single worker.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Where exactness can be traded for speed/cost, consider approximate algorithms: HyperLogLog for distinct counts, t-digest for quantiles, or Count-Min Sketch for heavy-hitter estimation. These give orders-of-magnitude lower memory and shuffle requirements and are often acceptable for dashboards or alerts. If exact results are required, make sure your combiner and merge logic preserve determinism and that your sink supports idempotent writes (so retries don't double-count).

Operationally, use a managed, autoscaling runner (Dataflow) and optimize worker sizing, shuffle memory, and the runner's shuffle parameters. Persist intermediate results to durable storage (partitioned Parquet on GCS) for very large jobs to enable restart-from-checkpoint and to avoid recomputing from raw sources on failures. Also implement monitoring for per-key distribution, shuffle bytes, and task skew so you can iterate on sharding, combiner logic, and cluster sizing.

Finally, plan for storage and consumption: write aggregated results into partitioned, clustered BigQuery tables or a time-series store designed for massive aggregations, and expose incremental outputs (per-window or delta updates) so downstream consumers don't have to scan massive tables repeatedly. Document the aggregation semantics, error bounds if approximate, and provide replayable pipelines so you can re-run aggregates when business logic changes.

30. You need to manage per-user state (e.g., session duration). Which Beam feature would you use?

I'd use Beam's stateful processing with keyed DoFns - specifically per-key StateSpec and TimerSpec inside a DoFn. Key the stream by user_id (or session_id), keep session attributes in per-key state (last activity timestamp, accumulated duration, event buffer) and use timers to emit or expire sessions after inactivity. This gives precise control over session semantics and works well with event-time processing and watermarks.

If the session definition matches activity gaps, session windows are a high-level abstraction that often fits: session windows automatically group events by inactivity gaps and integrate with watermarks, making many session-use cases trivial. For more custom behavior (multi-step enrichment, partial emissions, or complex merges) the stateful DoFn with explicit timers is the right tool because it supports arbitrary state shapes and controlled emissions.

Be mindful of state-size and retention: set TTLs for state entries and keep the per-key state compact. If you expect very large per-user state or long-lived sessions beyond runner state limits, persist snapshots to an external store (Bigtable, Redis) and load/cache portions as needed - but prefer the runner-native state for low-latency session handling when possible. Also design state updates to be idempotent (store last-processed event-id) so replays or retries don't corrupt session values.

Operational concerns matter: rely on a managed runner (Dataflow) that manages the state backend and checkpointing, monitor state size distribution and timer backlog, and tune worker resources accordingly. Test with time-advanced unit tests (virtual time) to validate timer behavior and edge cases like late events, and ensure sinks accept updates (MERGE semantics) so session finalization can overwrite provisional outputs.

Finally, document the session contract (how inactivity is defined, how late events are handled, what fields are stored) and provide runbooks for reprocessing sessions from raw archives. That makes the stateful design robust, auditable, and maintainable for long-running production workloads.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

31. You need to read data from Pub/Sub and write results into BigQuery. How would you build this?

I'd start with a simple, reliable ingestion path: Pub/Sub → Dataflow (Apache Beam). Pub/Sub gives durable, scalable ingestion and Dataflow is ideal for parsing, validation, enrichment, windowing and best-effort dedup before the sink. Keep the pipeline logic focused - parse the message envelope, assign event timestamps (WithTimestamps), validate schema, and route malformed messages to a dead-letter topic/GCS for later inspection.

Next, make writes to BigQuery idempotent and safe for retries. For low-latency use cases I'd use the Storage Write API (or BigQuery streaming with insertId) and set an insertId (derived deterministically from the event id or natural primary key) so at-least-once delivery doesn't create duplicates. For stronger correctness, stream into a staging partitioned table and run frequent MERGE jobs that upsert into the production table keyed by the primary key + commit timestamp - this handles updates/deletes and simplifies reconciliation.

Operational robustness matters: add metrics and monitoring (Pub/Sub ack lag, Dataflow worker health, processing latency, BigQuery errors), and implement retry/backoff for transient errors. Emit audit columns (source_topic, publish_ts, pipeline_version, batch_id) so you can trace any row back to the original message. Also persist raw messages in a GCS archive (or raw BigQuery table) as the canonical replay source if you need to reprocess with updated logic.

For schema and evolution, enforce a stable contract: include a schema_version in the message, validate early, and for additive fields either keep an extensions JSON or coordinate schema promotions through an approval process. Finally, automate tests (unit tests for Beam transforms and integration tests) and run a shadow mode comparison when first cutting over so you can verify parity before turning the pipeline authoritative.

32. You want to ingest Avro files from GCS. How would you set it up?

I'd treat Avro as a first-class bulk format and use BigQuery load jobs where possible because BigQuery natively understands Avro and preserves schema and nested types efficiently. Organize files in GCS with a deterministic layout (e.g., gs://bucket/path/date=YYYY-MM-DD/) and maintain a manifest or use Pub/Sub notifications to discover new files so you only process new arrivals.

Before loading, validate a sample of Avro files (schema compatibility, nullability, partition column presence). For daily loads I'd trigger a Data Fusion/Dataflow step (or a simple Cloud Function) to kick off a BigQuery load job per partition using bq load or the BigQuery API - load jobs are cost-effective for large files and avoid per-row streaming costs. If files are very large, split or let load jobs handle them; if you need transformation, run a Dataflow job that reads Avro, transforms, and writes optimized Parquet/Avro for BigQuery loads.

Make the process idempotent: record file metadata (filename, generation, checksum) in a manifest table and mark processed files only after successful load + validation. For schema mismatches route offending files to an isolate folder and capture errors (BigQuery load errors include row-level reasons). If downstream requires update semantics, load into a staging table and run a MERGE into the final table to dedupe and apply business rules.

Operationally add post-load validation (row counts, key-level checksums, and sample field checks) and alert on mismatches. Keep raw Avro files archived for replay and retention, and automate backfills by replaying the manifest. Document the file naming contract, expected Avro schema, and runbook for manual reprocessing so teams can recover and audit loads reliably.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

33. A streaming pipeline must write results into Bigtable with low latency. How would you design it?

For low-latency writes to Bigtable I'd use Pub/Sub → Dataflow → Cloud Bigtable. Dataflow gives you the ability to batch and parallelize writes, and the official Cloud Bigtable connector (or the client libraries in an async DoFn) lets you perform high-throughput, low-latency MutateRows (or BulkMutate) operations while respecting Bigtable best practices for key design and throughput shaping.

Design row keys carefully for write patterns: choose a key that avoids hotspotting (include a hashed prefix or time-bucket when necessary) while keeping lookups efficient for consumers. Use the Bigtable client's bulk mutation APIs with small, controlled batches (tunable: e.g., hundreds to low thousands of mutations per RPC depending on your instance size) and implement exponential backoff/retry for transient gRPC errors. For extremely latency-sensitive paths, keep batch sizes small to minimize per-record latency, but balance that against RPC overhead.

Make writes idempotent: include a deterministic row key + column timestamp or a version field so retries and replay don't create duplicate states. If you need conditional updates, use CheckAndMutateRow where supported, but remember that conditional operations are more expensive and can reduce throughput. Also tune Dataflow worker parallelism and leverage autoscaling so you meet peak load without overprovisioning.

Operational and reliability controls: monitor Bigtable CPU, storage, and hot-spot metrics, Dataflow worker CPU/memory and backlog, and system latencies. Use instance-level autoscaling or capacity planning, and colocate Dataflow workers in the same region to minimize network latency. Keep a raw event archive in GCS for forensic replay in case of downstream correction or reconciliation needs.

Finally, test at scale with representative keys and traffic shapes to find the sweet spot for batch size, parallelism, and instance node counts. Document key design, write semantics, expected latencies, and the reprocessing runbook so operators can scale or recover the pipeline without impacting downstream SLAs.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

34. Your team wants to pull change data capture (CDC) from Cloud SQL into BigQuery. How would you implement this?

I'd pick a managed CDC capture (Datastream) to avoid writing fragile tailing code - Datastream can capture the initial snapshot and then the ongoing binlog/stream of changes from Cloud SQL and publish them to Pub/Sub or GCS as a durable change log. That gives us an ordered, replayable feed and keeps load off the source DB, which is essential for production reliability.
Next, build a processing layer (Dataflow/Beam) that subscribes to the CDC feed, parses the CDC envelope (op-type, primary key, commit timestamp, before/after images), normalizes types, and enriches as needed. The job should produce a deterministic envelope: operation, key, commit_ts, payload, and source-offset so downstream steps can dedupe, order, and reconcile. Keep malformed records going to a DLQ for later inspection.
For applying changes to BigQuery, avoid blind streaming inserts. Stream changes into a partitioned staging table (or use Storage Write API with insertId for near-real-time) and then run deterministic MERGE statements into the production table keyed by primary key + commit_ts. This staging+MERGE pattern handles inserts/updates/deletes idempotently and supports replays without duplication.
Account for the initial snapshot: either let Datastream provide the snapshot or run a controlled export that loads the initial state into BigQuery staging, then resume applying changes from the CDC offset. Make sure the pipeline records the snapshot's ending offset so you don't miss or double-apply events. Also keep a raw archive of all CDC events in GCS for audits and reprocessing.
Finally, harden ops: monitor replication lag, Pub/Sub backlog, merge latencies and DLQ rates; implement schema-evolution handling (extension JSON or schema registry + alerts); secure connections with VPC/private connectivity and least-privileged service accounts; and create runbooks for replaying from offsets or repairing mismatches (row counts/checksums) so we can restore parity quickly if issues occur.

35. Schema mismatches occur between input and BigQuery sink. How would you fix them?

First, fail fast at validation: add an early schema-validation step in the pipeline that compares incoming records to an expected schema and routes mismatches (extra fields, missing required fields, type errors) to a dead-letter area with examples and reasons, so the main load isn't corrupted and you have artifacts to debug. Treat the raw input as canonical and don't mutate it until you've validated.
Second, implement an explicit schema-mapping/transformation stage that renames fields, casts types safely (with fallbacks or nulls), drops unsupported fields, and derives missing columns with sensible defaults. Perform these transformations in a controlled transform (Dataflow/Data Fusion) or as SQL in a staging BigQuery table so the output exactly matches the target schema before any final write.
Third, support additive/unknown fields by preserving them in an extensions JSON/STRUCT column or an auxiliary audit table. That way you don't lose data when producers add harmless fields and you can review those fields later to decide whether to promote them into the canonical schema. Keep schema changes explicit - a PR + migration - rather than letting pipelines silently evolve.
Fourth, for frequent or automated schema evolution, build a discovery + review workflow: detect new fields automatically, alert owners with sample values, and require an explicit approval to add columns to production. When promoting a column, add it as nullable with a default and run a backfill if needed; use CI tests to ensure downstream queries won't break.
Finally, make the sink writes idempotent and safe for retries: write to staging then MERGE into production, keep manifests (file/checksum/batch_id) to avoid double-processing, and add post-load validation (row counts, key checksums). Document mapping rules, maintain schema versioning, and provide runbooks for backfills and emergency rollbacks so schema mismatches are managed, auditable, and recoverable.

© Shubham Wadekar

36. Your source system produces Avro files. How would you leverage them in Dataflow?

I'd treat Avro as a first-class, schema-bearing input and read it with Avro-aware IO (use AvroIO / AvroIO.read() or the SDK's Avro support) so the pipeline gets strongly-typed records (GenericRecord or SpecificRecord) instead of raw text. That preserves the producer schema, keeps types intact for nested records, and makes downstream transforms simpler and safer.

Next I'd do an early validation and normalization ParDo: validate required fields, coerce or parse types that need it, add audit metadata (source filename, generation, ingestion_ts) and emit malformed rows to a dead-letter PCollection so a few bad files don't block the entire job. Because Avro carries schema, I'd also use it to detect schema evolution automatically and compare against an expected schema (side-input) to decide whether to accept, map, or quarantine a file.

For bulk historical loads prefer using BigQuery load jobs that accept Avro directly - i.e., have Dataflow orchestrate Load jobs into partitioned staging tables - because load jobs are cheaper and faster for large files. For streaming-like or record-by-record processing, convert GenericRecord → TableRow (or the language equivalent) in a transform and write with BigQueryIO.write() (Storage Write API or streaming inserts) using deterministic keys or MERGE patterns for idempotency.

Operationally, parameterize file patterns and use a manifest table to record processed file names/checksums so reruns are safe. Add post-load validation (row counts, a few aggregates) and a small sampling check that reads a few Avro records and compares them to the loaded rows. Keep raw Avro archived in GCS for replay and auditing.

Finally, take advantage of Avro's schema evolution: design mapping logic to handle additive fields (store extras in an extensions JSON if not mapping immediately), and include CI tests against sample Avro files so future producer changes don't silently break consumers. This preserves correctness while using Avro's strengths for robust, typed ingestion.

37. You must read JSON from GCS and convert it into BigQuery schema. How would you design this?

I'd begin with a robust ingest step that reads JSON lines from GCS using a text source (TextIO) and then parse each line in a ParDo/DoFn into a typed object or TableRow. Do the parsing early so you can validate structure, required fields, types and route malformed JSON to a dead-letter sink rather than failing the whole job.

Next, implement an explicit schema-mapping layer as a deterministic transform: rename fields, cast types (string→timestamp, string→numeric) with safe fallbacks, flatten nested objects into the target BigQuery STRUCTs or keep them as nested types if BigQuery schema supports it. Keep the mapping rules in a versioned config (side input) so you can update mappings without code changes and enable automated schema checks in CI.

For writes prefer a staging pattern: write parsed rows into a partitioned staging table (or temporary GCS Avro/Parquet files) and then run a controlled MERGE into the production table. This lets you dedupe, apply business rules, and handle non-nullable fields safely. If latency needs are low, use BigQuery load jobs from compacted Parquet/Avro files; if you need near-real-time, use Storage Write API with insertId or merge logic to avoid duplicates.

Operational concerns: emit metrics for parse success/failure counts, schema drift alerts when new fields appear, and maintain a manifest for processed files (filename, checksum, row-count) so replays are safe. Add a small post-load validator that compares source row counts and key aggregates to the BigQuery partition to detect partial loads early.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Finally, document the data contract (expected JSON schema, nullable fields, mapping rules) and provide a replay/runbook for fixing and reprocessing bad files. That makes JSON ingestion predictable, testable, and safe for downstream consumers.

38. Your team prefers Protobuf for streaming events. How would you implement this?

I'd start by making Protobuf the canonical message format end-to-end: define well-versioned .proto schemas, enforce compatibility rules (only additive changes, reserved field numbers for removals), and generate language-specific classes for producers and consumers so messages are strictly typed and compact on the wire.

In GCP, leverage Pub/Sub (or any broker) with Protobuf payloads and consider using Pub/Sub's native schema support so producers are validated at publish time. On the consumer side (Beam/Dataflow) use the generated Protobuf classes to parse bytes into typed objects in a ParDo (or use ProtoCoder/schema-aware coders where available) - this avoids brittle JSON parsing and gives you fast, memory-efficient decoding for high-throughput streams.

Once decoded, convert Protobuf messages into the pipeline's internal representation (or directly to TableRow for BigQuery), perform validation/enrichment (side inputs for lookups), and write to sinks. For BigQuery, either map the proto fields to BigQuery columns and use BigQueryIO (ensuring schema compatibility), or write to an intermediate store if more complex transformations or merges are required. Include an event_id, schema_version, and ingestion_ts in the envelope for idempotency and tracing.

Handle schema evolution by keeping a schema registry or by using the proto version field; implement graceful handling of unknown/new fields (store them in an extensions map or ignore if producers promise compatibility) and test backward/forward compatibility frequently. For very low-latency needs, tune batching and worker parallelism so protobuf decoding + writes meet latency SLOs; protobuf's compact binary format helps reduce serialization cost and network I/O compared with JSON.

Finally, add operational pieces: CI tests that validate producer/consumer proto compatibility, DLQ for messages that fail parsing or validation, monitoring for schema-version distribution, and documentation for how to roll out proto changes. This gives you a compact, efficient, and strongly-typed streaming pipeline that's safe to evolve over time.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

39. You're asked to validate schema consistency across pipeline stages. How would you enforce this?

Start by declaring a single source of truth for schemas - a schema registry or canonical schema files (Avro/Proto/JSON Schema) stored in source control. At design time, every transform, test, and deployment reads the canonical schema so the same contract is used from ingest -> transforms -> sink. This prevents "one-off" field name mismatches and gives you a single place to review changes.

Add automated contract checks into CI: unit tests that validate a pipeline's input/output Schema against the canonical one, and integration tests that run a small sample through the actual pipeline and assert the output schema and a few key transforms. Fail the build if a change isn't accompanied by a schema bump or a documented mapping - make schema changes an explicit PR process.

Enforce runtime validation in the pipeline: add an early schema-validation stage that checks each record against the expected schema and routes mismatches to a dead-letter store with clear reason codes (missing field, type mismatch, unexpected field). Emit metrics for schema violations so you get alerted immediately instead of discovering schema drift later in production.

Use schema-aware IO where possible (Avro, Protobuf, Beam Schema) so readers/writers do the heavy lifting and detect incompatibilities early. When sinks like BigQuery are involved, use a staged approach (write to a staging table first) and only promote to production after schema checks and reconciliation; this prevents broken downstream tables from appearing in production.

Finally, operationalize governance: require schema-version metadata in each message, keep a change-log of schema edits, run periodic scans comparing the canonical schema to what's actually observed in raw inputs, and add alerts for unexpected new fields. That combination - design-time contracts, CI checks, runtime validation, and governance - keeps schema consistency enforceable and auditable.

40. Source schema evolves with new fields added. How would you adapt Dataflow pipelines?

Treat schema evolution as an expected event and make ingestion tolerant: land raw inputs into a permissive staging (e.g., a raw JSON/Avro table or a raw_record column) so new fields don't immediately break downstream processing. That archive becomes your immutable source-of-truth for reprocessing and auditing.

Implement a controlled mapping step that reads the canonical schema (as a side-input/config) and applies explicit transforms: map known fields, cast types, and move unknown/additive fields into an extensions JSON/STRUCT column. That preserves new data for future use while keeping the production schema stable and predictable.

Automate detection and review: pipeline code should detect new fields in incoming records and alert the data owners with sample values. Make schema promotion a small, documented workflow - add the new field to the canonical schema, update mapping logic, run a backfill from the raw archive (or run a targeted reprocess for affected partitions), and then flip the production table after validation.

In Beam/Dataflow use schema-aware coders and robust parsing practices: use GenericRecord for Avro, Protobuf generated classes with optional fields, or Row/Beam Schema with safe get patterns. Design transforms to handle missing fields gracefully (provide defaults) and to accept extra fields without failure. If you plan to add columns in BigQuery, prefer adding them nullable and backfilling when appropriate.

© Shubham Wadekar

Finally, version everything and test: require a schema-version in messages, include compatibility checks in CI, run a shadow reprocess to validate the promoted field, and document the retention and governance policy for extensions so engineers can later promote useful fields safely. This makes schema evolution manageable, auditable, and low-risk.

41. A streaming pipeline has backlog building up. How would you diagnose scaling?

First, I'd check the obvious ingest-side metrics: is Pub/Sub (or whatever broker) backlog growing? Look at subscription backlog, publish and ack rates, and whether messages are piling up at the source. If the backlog is upstream, scaling the workers won't help until producers slow or you increase consumer parallelism/throughput.

Second, inspect the runner's processing metrics: worker CPU, memory, heap/GC, task slots, and I/O. High CPU/GC or frequent task retries indicate worker resource saturation. If workers are idle but throughput is low, the bottleneck may be upstream/downstream IO or a single-threaded transform. Correlate backlog growth with worker metrics to see which side is the problem.

Third, look at pipeline internals: check per-stage latencies, shuffle bytes, and element processing time. A hot key or skewed partition often concentrates work on a few bundles and causes backlog even when overall resources look fine. Also check if expensive external calls (DB/API) are blocking DoFns and throttling throughput.

Fourth, review autoscaling/worker config and quotas: is autoscaling maxed out (maxWorkers reached) or are there quota limits (CPU, IPs, API quotas)? If autoscaling is constrained, either raise maxWorkers, change machine types, or request quota increases. Also confirm whether you can use streaming optimizations (Streaming Engine, Dataflow Shuffle) that offload work from workers.

Finally, act with measured changes: if it's worker saturation, increase workers or machine size and tune parallelism; if it's skew, add salting/pre-aggregation or change keying; if it's external IO, add batching/caching or async calls. After each change, monitor backlog, processing latency, and error rates until the backlog clears and throughput stabilizes.

42. A batch job runs slowly due to insufficient workers. How would you adjust scaling?

Start by confirming the job profile: check stage-level durations, CPU/memory utilization, and shuffle/read/write bottlenecks so you know whether to add more parallel workers or larger workers. If executors are saturated (high CPU, spilling, or OOMs), you need more memory/cores per executor or more executors to increase parallelism.

Next, increase worker concurrency appropriately: either raise the number of workers (horizontal scale) or choose machines with more vCPUs/memory (vertical scale). For Spark-style jobs, tune executor count, executor cores, and executor memory to ensure you're not oversubscribing CPU or leaving resources idle; use the rule-of-thumb for cores-per-executor to balance GC and throughput.

If the cluster supports autoscaling, enable/adjust autoscaling policies so the cluster grows for heavy stages and shrinks when idle - this avoids permanently over-provisioning. For cost-sensitive runs, consider adding preemptible workers to give you more cores at lower cost while keeping a few non-preemptible workers for stability.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Also reduce unnecessary work before increasing capacity: repartition inputs to match the parallelism you intend, compact small files, push predicates down, and convert data to columnar formats (Parquet) to reduce IO and shuffle. Often these optimizations reduce worker needs dramatically.

Finally, validate changes on a representative sample and iterate: run a scaled test, observe job runtime improvement and resource utilization, then pick the smallest cluster configuration that meets your SLA. Document the sizing rationale so future runs use tuned defaults rather than guesswork.

43. Your job is hitting worker limits. How would you address horizontal scaling?

First, identify what "worker limits" means in context - reaching quota limits, node limits, or per-node resource exhaustion. If it's cloud quotas (VM count, CPUs, IPs), request quota increases or reorganize workload across regions/projects. If it's per-job limits like maxWorkers, raise that limit or enable autoscaling with a higher cap.

Second, reduce hotspotting and improve parallelism so you can make effective use of more workers. Fix skewed keys by salting hot keys, pre-aggregating heavy keys, or splitting large partitions; that prevents a few workers from doing most of the work while others sit idle. Repartition or coalesce strategically so you actually utilize added workers.

Third, consider changing the worker profile: use more smaller workers (horizontal scale with many nodes) when you have many independent tasks, or fewer larger workers when tasks are memory-heavy. Tune container/executor settings (cores per executor, memory) so each worker hosts an appropriate number of parallel tasks without frequent GC/OOM.

Fourth, break the job into stages or micro-batches that can run in parallel (fan-out/fan-in). Instead of one giant job, run multiple partitioned jobs (by date range, shard id) concurrently and then merge results; this often circumvents single-job scaling limits and makes retries/diagnosis easier.

Finally, add resilience and monitoring: implement graceful backoff for external services, ensure writes are idempotent for safe retries, and monitor per-worker throughput and shuffle bytes so you can detect when adding workers actually improves throughput versus when architectural changes are required.

44. You see uneven distribution of workload. How would you fix data skew?

First, diagnose precisely where the skew is coming from - inspect per-task/worker metrics (task durations, shuffle read/write bytes, CPU and GC), look for a few tasks doing much more work, and sample the keys those tasks handle. Quantify skew (e.g., top-1% keys account for X% of rows) so you know whether it's a few hot keys or many moderately-heavy keys; the remediation you pick depends on that profile.

Second, try algorithmic fixes that reduce the need to shuffle the heavy data: push filters and projections earlier, use map-side pre-aggregation or combiners (CombinePerKey), and prune or sample irrelevant columns before the expensive stage. Reducing the volume before shuffle often removes skew symptoms entirely or makes them manageable.

Third, apply sharding/salting for truly hot keys: append a small random or hash shard suffix to the key on the large side before the shuffle, perform the join/aggregation across salted keys in parallel, and then un-salt (reduce) the intermediate results into the final key. This spreads a single hot key across many workers and is cheap to implement if you handle the un-salt merge carefully.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Fourth, use hybrid strategies for very heavy keys: detect the heavy hitters and treat them separately - process the bulk of keys with the normal distributed flow and route hot keys to a dedicated path (precompute, cache, or use a keyed worker pool). For joins, broadcast the small side or write the small side to a fast lookup store instead of shuffling both sides.

Finally, validate and iterate: run a POC with representative data, monitor per-worker load and shuffle bytes, tune the number of shards/salt values, and add alerts for re-emergence of skew. Document the pattern and include skew-detection as part of pipeline telemetry so future data changes don't silently recreate the problem.

45. Your pipeline has a hot key issue. How would you redesign it?

Start by proving which key(s) are hot and why - measure per-key counts, identify temporal patterns (bursty vs steady), and check downstream data access patterns that expect single-key locality. Understanding whether the hotspot is inherent to business logic or an artifact of key design is crucial before changing semantics.

Next, change the key design or logical partitioning to distribute load: convert a monolithic key into a composite (e.g., add time-bucket, region, or hashed prefix), or shard the key space deliberately. For aggregation use-cases, salting (adding a small random or hash suffix) on write and a final un-salt aggregation is usually the most practical approach to parallelize a hot key without changing upstream producers.

Third, introduce pre-aggregation or an intermediate buffer for the hot key: accumulate events for that key in memory or a local aggregated store for a short window, emit a single summarized update downstream, and then persist the compacted result. This reduces write/read fanout and smooths bursts while preserving correctness if you implement deterministic merging.

Fourth, offload very-high-volume keys to a purpose-built store: put per-key state for heavy keys in a low-latency KV store (Bigtable, Redis) that's designed for hot single-key access, and use streaming writes directly to that store. Combine this with an async reconciliation job that compacts store state into analytical tables periodically so you don't overload analytical sinks.

Finally, ensure idempotency and observability: design merges/compactions to be idempotent (use transaction ids or versioned writes), add metrics for hot-key traffic, and test recovery and replay paths. Document the special-case handling so future engineers know which keys are routed to special logic and why, preventing accidental reintroduction of the hot-key bottleneck.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

46. A pipeline fuses multiple steps causing bottlenecks. How would you handle fusion optimization?

First, recognize why fusion is a problem: the runner (Beam/Dataflow or a Spark optimizer) has merged multiple transforms into a single execution unit, which can make bundles/containers do too much work, grow memory, and create long GC pauses or slow tail tasks. In interviews I say: treat fusion as an optimizer doing its job - then intentionally introduce logical materialization points where the optimizer should stop fusing. These act like short checkpoints so work is spread across stages.

Second, add explicit fusion-breaks. In Beam use Reshuffle() or a GroupByKey trick to force a materialize-and-read barrier; in Spark insert a repartition()/coalesce() or write/read to temporary storage (GCS/Parquet) for very heavy stages. These steps create smaller, independent units of work so you avoid huge monolithic bundles that cause memory/CPU hotspots.

Third, push combinable work earlier with proper combiners: use CombinePerKey (or Spark aggregations) before any wide join or global processing so you reduce the volume that subsequent fused transforms must handle. Pre-aggregation shrinks data and often removes the need for heavy fusions in the first place.

Fourth, simplify heavy DoFns/transforms: break a monolithic transform into smaller, single-responsibility transforms so each piece is easier to optimize and parallelize. Also remove expensive side-effects inside a fused transform (blocking I/O, long external calls) - move them to an async or externalized step so a single bundle doesn't block many records.

Finally, test and iterate: measure bundle durations, memory, and GC after each change. Use smaller POCs to experiment with where to place Reshuffle() or partitioning boundaries. Document the chosen fusion strategy so future devs understand why a materialization was added - it looks wasteful at first but pays off by stabilizing latency and reducing stragglers.

47. A job spends too much time shuffling. How would you optimize?

Start by understanding what causes the shuffle - large GroupByKey/CoGroupByKey, joins, or repartitions. I'd profile shuffle bytes and task times to find the worst offenders; optimization must be data-driven, not speculative. Once located, aim to reduce the amount of data that needs to be shuffled in the first place.

Apply pre-aggregation where possible: use CombinePerKey or map-side combiners to collapse records before the shuffle so fewer bytes move over the network. For joins, broadcast the small side (side-input) instead of shuffling both sides when one dataset fits in memory. Also prune unneeded fields early (projection/predicate pushdown) so you only shuffle the minimal payload.

Handle skew explicitly: detect hot keys and salt them (split into N subkeys) so their work is distributed across many tasks, then un-salt the results with a second aggregation. Alternatively, treat very-heavy keys with a special path (precompute, cache, or process them separately) to avoid overwhelming a single partition. For Spark, tune spark.sql.shuffle.partitions; for Beam, tune runner-specific shuffle/worker settings if available.

Optimize I/O and formats: convert inputs to columnar formats (Parquet/ORC) so you read and transmit less data; compact small files to avoid many tiny tasks; and prefer server-side load jobs (BigQuery load) for bulk transfers rather than per-record streaming. Finally, validate changes with a scaled test run - measure shuffle reduction and end-to-end runtime - and iterate until the shuffle no longer dominates job time.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

48. Your pipeline suffers from hot key aggregation. What strategies would you apply?

First, quantify which keys are hot and how much they dominate traffic - you need that to pick the right mitigation. Classic fixes start with sharding/salting: append a small shard id to the hot key (e.g., key#0..N-1), aggregate in parallel across shards, and then un-salt by reducing the N partial aggregates back into the original key. This spreads load while preserving exactness.

Second, pre-aggregate upstream if possible: perform map-side or incremental aggregation close to the producer (edge, collector, or worker) so fewer events reach the global reducer. For streaming, use per-key local buffers/state to emit periodic summaries rather than every event - it smooths bursts and reduces pressure on the global aggregator.

Third, offload very-hot keys to a specialized store: route those keys to a low-latency key-value service (Bigtable, Redis) for point updates and keep analytics on compacted snapshots. This is especially effective when consumers need point-reads/updates rather than full-table analytics and avoids hammering distributed shuffles.

Fourth, use approximation when business allows it: replace exact aggregation with approximate algorithms (HyperLogLog for uniques, Count-Min Sketch for counts) that are far cheaper and distribute nicely. If exactness is required, combine approximation for monitoring and exact paths for reconciliation only when thresholds are crossed.

Finally, implement monitoring and graceful degradation: alert on hot-key spikes, dynamically increase shard count for problematic keys, and provide fallbacks (throttling, delayed processing) for non-critical consumers. Document the special-case handling for the hot keys so the team remembers why the design departs from the generic aggregation pattern.

49. Latency is high in streaming mode. How would you leverage the Streaming Engine?

First, I'd explain what Streaming Engine gives us: it offloads shuffle, state and timers from worker VMs into a managed service layer so workers spend far less time on heavy coordination and GC. In practice that reduces per-element latency and removes some sources of tail latency because the service handles redistribution and checkpointing more efficiently than worker-local shuffle. Enabling it is often the quickest win for high-latency streaming jobs.

Second, the practical steps I'd take: enable Streaming Engine when submitting the Dataflow job (set the streaming engine flag in job options), ensure the Beam SDK and runner version used are compatible, and test in a staging environment. After switching it on, I'd watch worker CPU/GC metrics - they should drop - and measure end-to-end processing latency and watermark progress to confirm the expected improvement.

Third, Streaming Engine works best when you pair it with other streaming optimizations: reduce heavy per-record external I/O in DoFns (batch or async external calls), use Combine/CombinePerKey to pre-aggregate before shuffles, and fix hot-key skew (salting or separate handling). The Engine helps, but you still need to remove blocking work from the critical path so the managed shuffle isn't waiting on slow RPCs.

Fourth, be mindful of compatibility and trade-offs: verify any custom IOs or native connectors you use are supported with Streaming Engine, and validate that autoscaling behavior remains acceptable (you may need to adjust maxWorkers or streaming parameters). Also measure cost implications - while worker CPU goes down, you're using a managed service tier that shifts where costs appear; run a short A/B test to quantify cost vs latency gains.

Finally, operationalize the change: add latency and watermark dashboards, run chaos tests (simulate bursts/late data) to ensure stability, and keep a rollback plan in CI so you can revert if a dependency isn't compatible. Document the decision and the measured improvements so stakeholders see the latency benefit and any cost tradeoffs.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

50. A pipeline processes many small files from GCS. How would you optimize ingestion?

First, stop per-file one-by-one processing at the source: use GCS notifications (Pub/Sub) or a manifest table to detect new objects instead of repeatedly listing directories. That avoids expensive list operations and gives you exact file names to process so you don't waste cycles re-scanning the bucket.

Second, batch and compact small files before heavy processing. In a Dataflow/Data Fusion step, collect a group of small files (by time window or count threshold), read them in memory/stream, and write a single larger Parquet/Avro file partitioned appropriately. Consumers then read fewer, larger files which greatly reduces task startup overhead and shuffle overhead downstream. Aim for target file sizes (100MB–1GB) depending on your query patterns.

Third, implement write-side controls so producers don't emit many tiny objects in the first place: buffer and flush after size/time thresholds, write to a staging path and then atomically move/rename to the inbound path, or have producers write to day/hour partitions directly. If you can't change producers, run a periodic compaction job (Dataproc/Dataflow) that consolidates small files into optimized columnar files.

Fourth, tune the pipeline's parallelism and IO settings: coalesce partitions before writing, set appropriate number of output shards, and use efficient file formats and compression to reduce I/O cost. When using Spark/Dataproc, choose repartition/coalesce wisely; in Beam, control the number of output shards and use FileIO sinks that support large block writes.

Finally, add operational guards: maintain a manifest of processed files (name, generation, checksum) to avoid reprocessing, emit metrics for average file size and files-per-minute, and alert when small-file rates exceed thresholds. Provide a safe replay/compaction runbook so operators can consolidate historical small files without data loss. These steps cut the small-file overhead and make downstream analytics much faster and cheaper.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

51. Some messages fail due to transient API errors. How would you handle retries?

Start with exponential backoff and a capped retry budget at the call site - retry transient 5xx/429 errors with increasing delays and a maximum number of attempts so you don't thrash the downstream API or exhaust resources. Implement jitter to avoid thundering-herd retries when many workers see the same transient outage.

Make retries idempotent: ensure each retryable operation is safe to run multiple times (use idempotency keys, deterministic operation ids, or upsert semantics). If the external API can't guarantee idempotency, add a local dedupe / commit log that tracks operation ids so you can safely ignore duplicate attempts after a successful write.

Separate transient vs permanent failures: detect error classes (rate-limits, timeouts, transient 5xx versus client-side 4xx like bad request) and only retry the transient categories. For transient-but-prolonged failures route the message to a retry topic with exponential backoff (a retry queue pattern) and only move to the dead-letter queue after retries are exhausted.

Keep retries out of hot path where possible: use async batching and background workers to retry without blocking the main pipeline threads. That reduces latency for healthy messages and gives more control over retry throughput and backoff behavior.

Finally, monitor and alert on retry metrics (retry count, success-after-retry, messages moved to DLQ) and expose runbooks for manual intervention. If retry rates spike, treat it as an incident - investigate upstream causes (API outage, quota exhaustion) rather than blindly increasing retry budgets.

52. You need to capture failed records for reprocessing. How would you design a DLQ?

Treat the dead-letter queue as a first-class, durable artifact with rich metadata. For each failed record store the original payload, pipeline id/version, error code/reason, timestamp, processing attempt count, and any relevant provenance (source file name or Pub/Sub message id) so reprocessing or manual triage has full context.

Choose a durable sink and retention policy: short-term DLQ topic for immediate replays (Pub/Sub dead-letter topic) plus a longer-term immutable archive in GCS or BigQuery for audit and forensic work. Partition the archive by ingestion date and error type so auditors can query relevant slices quickly.

Design a replayable reprocessing flow: a small consumer or job that can read DLQ entries, apply automated fixes or transformations, and publish to a reprocess topic or staging table with idempotent semantics. Include tooling for manual review (view sample records, apply one-off fixes, requeue) so engineers and data stewards can handle tricky poison data safely.

Add lifecycle and governance: define who can read/clear the DLQ, retention/archival policies, and SLAs for how quickly DLQ items must be investigated. Instrument metrics and dashboards (DLQ growth, oldest message age, top error reasons) and set alerts when age or volume exceed thresholds so the DLQ doesn't become an ignored graveyard.

Finally, make reprocessing safe: ensure replayed messages carry the same ids and batch metadata so sinks can dedupe, keep audit trails for any corrected records, and run post-replay validations (counts, checksums) before marking issues as resolved. That gives confidence that reprocessing restores correctness without side effects.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

53. Poison-pill messages are crashing your job. How would you detect and handle them?

Detect poison-pill messages early by adding defensive validation and bounded try/catch around risky transforms: parse faults, schema mismatches, or exceptions from UDFs should be caught per-record and routed to the DLQ rather than bubbling up to crash the worker. Instrument counters for exception types and sample offending payloads for rapid diagnosis.

Use circuit-breaker behavior for repeated failures: if a single message repeatedly causes the same transform to throw even after safe parsing and minor sanitization attempts, quarantine it automatically (move to DLQ) and mark the pipeline to skip similar messages until a human inspects the failure. That prevents one bad record from taking down the whole job.

Isolate dangerous logic into smaller, testable units and run unit tests that include edge-case payloads. In streaming, use bundle-level safeguards and timeouts so a single record that causes excessively long processing can be abandoned and recorded instead of stalling worker threads. For external calls, use bounded retries and fallbacks so a slow external dependency on one record won't block others.

Keep an immutable archive of raw inputs so you can replay and debug poison pills offline. When you fix the root cause (parser bug or producer bug), use your replay tool to reprocess the quarantined records into the pipeline with the corrected logic, ensuring idempotency so duplicates don't occur.

Finally, operationalize detection with alerts and dashboards: notify on spikes of per-record exceptions, show top offending error messages and sample payloads, and keep a runbook for triage that includes how to inspect, fix, and safely replay poison-pill records. This turns poison-pill handling from a surprise crash into a repeatable remediation process.

54. Permanent errors must be separated from transient ones. How would you implement this?

Start by defining clear error classes and detection logic up front - what counts as transient (timeouts, 429/5xx, network blips) and what counts as permanent (schema violations, missing required fields, authentication errors). Encode this classification in the pipeline's error handler so each caught exception is mapped to a category immediately (don't rely on ad-hoc string matching; use error codes/types where possible).

Implement layered retry behavior for transient errors: local retries with exponential backoff + jitter inside the worker, then push to a retry queue/topic if still failing (with increasing backoff tiers), and only move to a dead-letter path after the retry budget is exhausted. Keep the retry logic configurable (retry counts, backoff windows) so you can tune for API SLAs without changing code.

For permanent errors, fail fast to a permanent DLQ and include rich metadata (original payload, pipeline version, error code, stack trace, attempt count, timestamps). Store permanent errors in a durable audit store (GCS or BigQuery) that data stewards or product owners can review - don't let permanent failures consume retry budget or block healthy traffic.

Add automated triage and circuit-breaker patterns: if a high volume of permanent errors of the same type appears, auto-alert and optionally hold similar incoming records (quarantine) until humans investigate; for transient error storms, escalate via throttling or backpressure rather than infinite retries. This prevents one failing downstream API or a broken producer from cascading.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Finally, make reprocessing and governance straightforward: provide a replay tool to take DLQ items (both transient and permanent after fix) back through the pipeline, track provenance and resolution status in a ticketable audit log, and emit metrics/alerts for both DLQ growth and oldest-message-age so the team treats DLQs as operational-first artifacts rather than ignored queues.

55. Duplicate records appear in sinks after retries. How would you ensure idempotency?

Design idempotency into the data model and sinks from the start: every event should carry a stable, unique event id (UUID, composed primary key, or producer-assigned sequence) and a commit/version timestamp. Use that id as the dedupe key downstream so any repeated attempts can be detected and ignored or merged deterministically.

Prefer idempotent write patterns on the sink: use upserts/MERGE semantics (BigQuery MERGE by primary key), the BigQuery Storage Write API with deduplication via insertId, or conditional writes (CheckAndMutate) for key-value stores. Avoid blind INSERTs for critical tables - writes should be replaceable or mergeable so retries don't create duplicates.

If exact once isn't supported end-to-end, add a dedupe stage in the pipeline before final writes: maintain a small stateful dedupe window (per-key state or a dedupe cache keyed by event id) that filters repeat events within a retention window, and persist processed event ids (manifest or dedupe table) for longer-term idempotency across restarts. Ensure the dedupe store is itself durable and scalable (BigQuery table, Bigtable, or Redis with persistence).

Make producer-side guarantees where possible: encourage producers to provide monotonically increasing sequence numbers per partition/topic and let the consumer reconcile out-of-order deliveries deterministically. Combine producer ids with consumer-side dedupe for the strongest guarantees.

Finally, test and monitor: create chaos tests that replay messages and simulate retries, instrument duplicate-detection metrics and alerts, and keep a reconciliation job that periodically compares source and sink row counts/checksums to find any missed duplicates. Document the idempotency contract so both producers and consumers know the expectations for retries and replays.

56. A pipeline shows backlog growing in Pub/Sub input. How would you debug?

First, I'd confirm the surface metrics: check the Pub/Sub subscription backlog (oldest unacked message age and total outstanding messages) and correlate it with Dataflow/consumer metrics. If backlog is growing but publish rate is low, the issue might be upstream; if publish rate is steady/high and backlog grows, the consumer is the likely bottleneck.

Second, inspect the consumer (Dataflow) health and throughput: look at processing latency, elements/second, worker CPU/memory, GC times, and number of active workers. If workers are saturated or GC-ing heavily you need more worker capacity or different machine types; if workers are idle but throughput is low, look for blocking operations (sync external calls, DB lookups) inside DoFns.

Third, examine operational and quota signals: check Dataflow autoscaling activity (are you hitting maxWorkers?), GCP quotas (VM, APIs), Pub/Sub push/pull error rates, and any throttling on downstream services (BigQuery ingestion quotas, external API rate limits). Quota or throttling issues often look like consumer slowdowns even when worker resources seem adequate.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Fourth, look for data-specific causes: hot keys/skew that concentrate work on a few bundles, very large records that slow processing, or a sudden increase in malformed records that are sent to DLQ with expensive handling. Sample failing/slow records and run a small repro job to isolate record-level issues.

Finally, act iteratively: if it's resource saturation, increase workers or tune parallelism and GC; if it's blocking IO, batch or async external calls or cache lookups; if it's quotas, request increases or redesign writes; if it's skew, apply salting or special-case heavy keys. Re-check backlog after each change and keep a short runbook for rolling recoveries (scale up, pause inputs, drain, replay) so you can clear backlog safely.

57. You want to measure throughput for a Dataflow job. How would you monitor it?

I'd instrument both runner-provided and business-level metrics. Start with Dataflow's built-in metrics in the UI: elements/second, system lag, per-transform throughput, and worker resource metrics (CPU, memory, disk, network). These give immediate visibility into how many elements are processed and where bottlenecks sit.

Next, export metrics to Cloud Monitoring and build dashboards that show end-to-end throughput (input rate vs processed rate), per-stage throughput and latency percentiles, and sink write rates (e.g., BigQuery rows/sec). Correlate throughput with worker autoscaling events and Pub/Sub publish/ack rates so you can see whether throughput changes are due to input, processing, or output constraints.

Add business-level telemetry inside your transforms using Beam's Metrics API (counters and distributions) to measure domain-specific throughput - for example "valid events/sec", "enriched events/sec", or "failed events/sec". Those counts appear in the Dataflow UI and Cloud Monitoring and make it easy to track the meaningful throughput for SLAs.

Also capture derived metrics like bytes/sec, average processing time per element, and shuffle bytes - these help explain why throughput is what it is (I/O-bound, CPU-bound, or network-shuffle bound). Finally, set alerts on trends: sustained difference between input and processed rate, increasing system lag, or worker CPU > 80% for sustained periods so you're proactively notified when throughput degrades.

58. You need to track custom metrics for business KPIs. How would you add counters?

I'd add metrics at the point in the pipeline where the business event is observable (inside the relevant DoFn/transform). Use Beam's Metrics API (Counters, Distributions, Gauges) - create a named counter in the transform, increment it for each business event (e.g., kpiOrdersProcessed.inc(1)), and emit distributions for sizes/latencies if helpful. Keep names stable and include a metric prefix that denotes the pipeline and KPI domain.

Make sure those metrics are captured by the runner: Dataflow surfaces Beam metrics in the job UI and exports them to Cloud Monitoring automatically. From Cloud Monitoring you can create dashboards, charts and alerting policies on your KPI counters (e.g., orders/sec, failedOrders/day) and correlate them with system metrics like latency and worker count.

For more complex KPIs (per-tenant, per-country), either use labeled counters (if your monitoring supports labels) or maintain light-weight side outputs that batch per-key summaries into a BigQuery KPI table. That lets you build historical KPI dashboards and run analytics on trends without overloading the monitoring system with cardinality explosion.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Finally, include tests and documentation: unit-test transforms to assert counters increment for sample inputs, add integration smoke-tests that validate metrics appear in Cloud Monitoring, and document metric names/semantics so analysts and SREs know what each counter means and how to interpret them. This keeps KPI metrics reliable, discoverable, and actionable.

59. A job appears stuck but is not failing. What steps would you take?

First, I open the job in the runner UI (Dataflow console or whatever orchestrator) and check the high-level indicators: watermark progress, backlog/lag, per-transform throughput, and worker health. These tell me whether work is slow because of input lag, stalled workers, or a long-running transform stage - that separates control-plane issues from worker-side bottlenecks quickly.

Second, I check logs and task-level metrics for the slow stages: look for repeated retries, long GC pauses, thread dumps, or synchronous external calls blocking DoFns. I also inspect the most-recent task logs on a few workers to see stack traces or repeated error patterns that don't surface as failures (e.g., timeouts with retries that keep the task alive but stalled).

Third, I validate external dependencies and quotas: confirm Pub/Sub subscription ack rates, BigQuery quotas, downstream API throttling, and any network or service errors. Often a job "stuck" is waiting on a slow external system or has hit a quota and is backing off silently - knowing that prevents pointless code changes.

Fourth, check autoscaling/worker conditions and system resources: is maxWorkers reached, are workers all evicted/preempted, are there many stragglers due to hot keys or skew, or is there a persistent GC/OutOfMemory pattern? If workers are resource-starved, I either increase autoscale caps/worker size or pinpoint skew and add salting/pre-aggregation so more workers can make progress.

Finally, act carefully: if the pipeline state is safe to restart, I drain and restart the job after addressing the root cause (fixing a blocking API, raising a quota, or patching a bug). If not, snapshot the state (or archive raw inputs), apply a hotfix in a staging run, and use a controlled cutover. Throughout, document findings and update the runbook so the next incident is faster to resolve.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

60. Memory usage is climbing over time in workers. How would you debug memory leaks?

First, confirm it's a real leak and not normal load: plot JVM/worker heap over time, GC frequency/duration, and number of active bundles/tasks. A leak will show steady heap growth between GCs and growing live set sizes; transient spikes that recover after GC are different and point to tuning rather than leaks.

Second, collect diagnostics from a problematic worker: enable verbose GC logs, grab heap/heapdumps (jmap/jcmd or equivalent) and thread dumps at intervals, and capture executor-level metrics (off-heap usage, native memory). Comparing heap dumps over time shows which classes/objects are accumulating and where references originate.

Third, inspect the code paths that run in workers: look for large static caches, unbounded maps, accumulating lists inside DoFns, or state that never TTLs. Common culprits are user-land caches, accumulators stored in static fields, third-party libraries that keep references, or retaining whole messages while batching. If using stateful processing, check that state TTLs are set and that state cleanup runs.

Fourth, narrow with targeted repros: run a small controlled job that exercises suspect transforms with synthetic data and confirm the leak reproduces. Patch the code (release caches, use weak/limited caches, fix accidental static references, close clients), rebuild, and retest. If a third-party library is to blame, upgrade or replace it and report upstream.

Finally, mitigate operationally while fixing: add worker restart windows or use autoscaling with shorter lifetimes, adjust container memory limits to fail-fast rather than OOM unpredictably, and add alerts for memory growth trends. Document the fix and add unit/integration tests that catch similar regressions early.

61. Your team wants to reuse pipelines with parameterized configs. How would you use templates?

I'd explain that Dataflow templates are designed exactly for this use case - separating pipeline code from runtime parameters so non-developers can launch jobs without changing code. I'd build the pipeline as normal in Beam, but instead of hardcoding things like input path, output table, window duration, I'd declare them as pipeline options (--input, --output, --windowDuration).

Then I'd compile the pipeline into a *classic template* (--templateLocation=gs://.../template.json). At runtime, anyone can launch the job with gcloud dataflow jobs run and provide different parameter values. That way, the same pipeline binary is reused for dev/test/prod simply by changing parameters.

This helps in CI/CD as well - I only build the pipeline artifact once, store it in GCS, and then deploy new jobs by providing environment-specific configs. It reduces errors, makes deployments repeatable, and empowers analysts or ops teams to launch jobs safely.

If the pipeline requires complex structures (multiple inputs/outputs, custom logic), I'd provide JSON parameter files or wrap the job submission with a script that injects the right configs for the environment. This pattern keeps flexibility while avoiding recompile/redeploy cycles for every small change.

Finally, I'd mention operational benefits: templates integrate with Composer/Airflow or Cloud Scheduler for automation, they support parameter validation, and they make reprocessing simpler since you can launch the same pipeline with a different date or input path without touching code.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

62. You must deploy a job that includes custom libraries. How would you use Flex templates?

Flex templates solve the limitation of classic templates, where you're restricted to pre-built SDKs. If I need to bundle custom code, third-party libraries, or non-Java/Python dependencies, I'd use a Flex template. That means I package my pipeline in a Docker container that includes the Beam SDK, my pipeline code, and any custom libraries or binaries.

The process is: create a Dockerfile that installs runtime dependencies (e.g., Python wheels, shared libraries, internal JARs), copy in the pipeline code, and set the entrypoint to run the Beam job. Build and push the image to Artifact Registry or GCR. Then generate a Flex template spec (JSON) pointing to that container image and upload it to GCS.

At job submission, Dataflow pulls that container, runs the pipeline, and scales workers with the same image. This guarantees consistency across dev/test/prod because every worker runs the exact same environment. It also means I can use non-standard dependencies or language versions that aren't available in classic templates.

Flex templates also support parameterization like classic templates, so I still provide runtime parameters (input, output, date). The main difference is packaging and deployment flexibility. In practice, I'd document the build/run steps and add the Docker image build to CI/CD so pipeline updates automatically produce new Flex template versions.

This approach is the go-to whenever I need custom connectors (like internal APIs), machine learning libraries, or any non-standard runtime requirements. It future-proofs the deployment and avoids the "works on my machine" problem.

63. Your company uses multiple programming languages. How would you use portable pipelines?

I'd explain that Beam's portability framework lets you write a pipeline in one SDK (say Python) and run it on a runner implemented in another language (like Java/Dataflow). That's possible because Beam introduced a *Universal Local Runner (UDF portability layer)* where transforms and DoFns can cross language boundaries using containerized SDK harnesses.

For example, if most of my team codes in Python but we want to leverage Dataflow's mature Java runner, I'd write the pipeline in Python, enable portability, and let Dataflow spin up Python SDK workers in containers. The runner orchestrates execution, while the SDK container executes user code - so the two languages interoperate seamlessly.

If I need a polyglot pipeline (e.g., most transforms in Java but one ML model inference step in Python), I'd use cross-language transforms. That means I can call a Python transform from a Java pipeline by depending on the portable expansion service. This avoids reimplementing logic in multiple languages and allows each team to work in the language they know best.

Practically, this requires using portable job submission (--experiments=use_runner_v2 on Dataflow, for example), ensuring the right SDK containers are published (Python, Go, Java), and coordinating schema so cross-language boundaries are consistent.

Finally, I'd highlight the business value: portability future-proofs our investment - if tomorrow we want to run Beam jobs on Flink or Spark instead of Dataflow, or switch languages, we don't have to rewrite everything. It decouples pipeline logic from runtime, and it lets us use the right language for the right task while still running on a unified runner.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

64. A CI/CD pipeline must deploy Dataflow jobs automatically. How would you implement this?

Start with a single source of truth: keep pipeline code, tests, and deployment configs in Git. Put small, fast unit tests and Beam SDK unit tests in CI so every PR validates transforms and schema logic. Build steps should produce a repeatable artifact - for classic templates that's a template JSON in GCS; for Flex templates that's a Docker image pushed to Artifact Registry plus a template spec - and store the artifact with a versioned tag (git SHA).

Use an automated build system (Cloud Build, GitHub Actions, or your CI) to run the pipeline: on merge to main build the artifact, run integration smoke tests against a dev environment, and then publish the template/image. Encapsulate infra and environment settings as code (Terraform / Deployment Manager) so dataset/table creation, service accounts, and IAM are reproducible and reviewed. Keep environment-specific runtime parameters out of code - pass them at deploy time via YAML/JSON config or Secret Manager for credentials.

Wire a deploy pipeline with promotion gates: automatically deploy to dev after successful build; for staging/prod require automated integration tests + a manual approval step. Use Cloud Build triggers or a CD tool to submit Dataflow jobs from templates (or start them via the REST API) with environment parameters, and run a post-deploy smoke test (small job or sample data) that validates end-to-end writes and basic metrics before marking the release healthy.

Make deployments observable and reversible: tag jobs with pipeline-version metadata, emit a health-check metric after startup, and record deploy events in an audit log. For Flex templates, bake the runtime image in CI to ensure parity. Implement automatic rollback: if smoke tests fail or SLA alerts trigger within a configured window, the pipeline should drain the new job and either re-launch the previous template or switch traffic back to the old job (see blue/green pattern below).

Finally, automate governance: require PRs for any schema or pipeline changes, run contract tests that validate input/output schemas, and capture cost estimates in CI. Keep a simple operator runbook (how to replay, drain, rollback) and integrate alerts into on-call channels so deployments are safe, auditable, and repeatable.

65. A running pipeline needs updates with minimal downtime. How would you handle it?

Treat streaming and batch separately. For batch jobs, rolling updates are trivial - stop the scheduled job and start the new version. For streaming jobs, prefer a zero-or-low-downtime strategy: either use Dataflow's supported streaming-job update mechanism for non-breaking changes (if your change is compatible), or perform a blue/green style deployment where you bring up the new pipeline in parallel and switch traffic gradually.

Blue/green approach: deploy the new job (green) pointing at the same raw changelog/archive but either a different staging table or a different Pub/Sub subscription. Run both in shadow mode: validate green's outputs against the old (blue) using sample checksums or row counts. When parity and smoke tests pass, atomically switch consumers (e.g., change the downstream dataset pointer, swap a pub/sub subscription alias, or update a feature-flag) so readers point to green. Then drain the blue job gracefully so it finishes in-flight work before shutdown.

If you need in-place updates, keep changes backward-compatible: preserve input/output schema, add feature flags, and try to limit breaking structural changes. Dataflow supports some streaming job updates in-place when the operator/runner allows it, but those updates must be tested thoroughly in staging - not all code changes or state-schema changes are safe. Always validate that stateful transforms remain compatible or provide a migration that rehydrates state from the immutable raw archive.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Make state and reprocessing first-class: persist raw inputs in an immutable store (GCS/BigQuery) so you can re-run or backfill partitions if the new job's logic needs correction. Use idempotent sink patterns (MERGE/upsert, insertId) so dual-writing during the swap won't create duplicates. Run reconciliation after cutover (row counts, checksums, key-level sampling) to confirm no data loss or divergence.

Finally, automate the rollout: use CI-built artifacts and parameterized templates, run automated smoke validations, monitor health (latency, watermark progress, DLQ rate) during the cutover window, and have a scripted rollback to drain the new job and restore the previous one if anything goes wrong. Document the exact update playbook so updates are predictable and minimize downtime.

66. A developer needs permission only to launch pipelines but not manage others. Which IAM role would you give?

Give them the **roles/dataflow.developer** role scoped to the environment/project where they should launch jobs. That role allows creating and running Dataflow jobs (including launching templates) without granting full admin rights to modify Dataflow templates, instance configs, or project-wide settings. It's the right balance for "can run but not manage."

Complement that with resource-scoped read access they need to actually launch: roles/storage.objectViewer (or a bucket-level binding) on the GCS location that holds templates/artifacts, and read access to any config secrets they must reference (Secret Manager roles/secretmanager.secretAccessor scoped to the specific secrets). Don't give broad Storage or Secret permissions - limit to the specific buckets/secrets used for running jobs.

If the pipeline uses other sinks/sources (BigQuery, Pub/Sub, Bigtable), grant only the minimal dataset/topic/table-level roles required for the job to succeed (for example BigQuery Data Editor on a specific dataset or Pub/Sub Publisher/Subscriber only on the topic/subscription the job needs). Prefer resource-level bindings rather than project-wide roles to avoid lateral privilege.

Finally, enforce environment separation: give the developer the Dataflow Developer role in dev/test projects but not in staging/production, and require an approval/CD pipeline for production launches. Document the expected launch workflow so they don't need elevated rights to do their day-to-day tasks.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

67. Pipelines are failing due to insufficient service account privileges. How would you fix this?

First, diagnose the exact permission failure: inspect pipeline logs and the error message (it usually names the missing permission, e.g., bigquery.datasets.update or storage.objects.create). Don't guess - the precise error tells you which permission is missing and on which resource, which keeps fixes minimal and safe.

Second, grant least-privilege permissions scoped to the resource. Map the needed permission to a predefined role when possible (for example roles/bigquery.dataEditor on a specific dataset, or roles/storage.objectCreator on a specific bucket). If no predefined role fits, create a custom role containing only the required IAM permissions and bind it to the service account. Avoid giving broad roles like Editor or Owner.

Third, apply the binding at the narrowest scope that works (dataset, bucket, topic) and then re-run the failing job to validate. If the job still fails, re-check logs for the next missing permission - complex pipelines often require a few granular permissions (e.g., BigQuery write + KMS decrypt + Storage write). Iterate until the job succeeds.

Fourth, prevent recurrence by adding a permissions manifest to the pipeline repo that lists service-account permissions required for each environment and by adding a smoke test in CI that runs a representative pipeline step using the service account. That way missing rights are caught at deploy-time, not in production.

Finally, document the change and audit it: record who approved the permission change, how long it will remain, and whether it's temporary (e.g., for a migration). If you granted elevated rights for a migration, schedule their revocation after the work is done and enforce time-bound IAM conditions where appropriate.

68. You must encrypt data at rest using CMEK. How would you configure this?

CMEK is configured per-GCP resource - you don't "turn it on" inside Data Fusion itself; you configure Cloud KMS keys and point each target resource (BigQuery dataset, GCS bucket, Pub/Sub topic, Dataproc disk) to use that key. Start by creating a KMS key ring and a crypto key in the same region as your data, and enable rotation if required by policy.

Grant the pipeline/service account the KMS role it needs on that key: roles/cloudkms.cryptoKeyEncrypterDecrypter (for encrypt/decrypt) or more restricted roles if you separate duties. Without this role, writes to CMEK-protected resources will fail at runtime, so test key access in a dev environment before production rollout.

Then configure each resource to use the key:

- For BigQuery: create or update the dataset with the KMS key as the CMEK key. BigQuery will then encrypt table data at rest with that key.
- For GCS: set the bucket to use CMEK by default or specify the key when writing objects.
- For Pub/Sub/Dataproc etc.: configure CMEK on topics or cluster disks respectively (Dataproc initialization/profile can set CMEK for ephemeral disks).
  For Dataflow/Data Fusion flows, ensure intermediate artifacts (Dataproc disks, GCS temp buckets, staging locations) are also covered by CMEK where required.

Operationally, test end-to-end: confirm the service account can write/read the CMEK-protected resources, validate logs for KMS usage, and verify you can rotate or disable keys according to policy. Lock down KMS IAM tightly (few principals can manage keys), enable audit logging for key usage, and document key lifecycle and regions so compliance reviewers can confirm encryption is enforced across the pipeline.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

69. Your company requires VPC-SC. How would you run Dataflow jobs inside a perimeter?

Keep everything the job needs *inside the perimeter* and avoid public egress. Put Dataflow workers in a subnet that belongs to the VPC service-perimeter project and set the pipeline to use that subnetwork (pass the --network / --subnetwork flags or set them in the Dataflow template/profile). Use Private Google Access / Private Service Connect patterns so workers talk to Google APIs (Cloud Storage, BigQuery, Pub/Sub) without leaving the perimeter. In practice that means using private endpoints or the Google-managed private paths that VPC-SC requires rather than public IPs for worker VMs (--use_public_ips=false where supported).

Make sure all external resources the job touches (GCS buckets, BigQuery datasets, Pub/Sub topics, KMS keys, Dataproc clusters, etc.) are also inside the same service perimeter (or exposed via an approved ingress path). If some managed service must remain outside, front it with a Private Service Connect endpoint or move the resource into a perimeter project. Don't rely on wide open network egress or IP allowlists - VPC-SC works by scoping service APIs themselves.

Use dedicated, minimally-privileged service accounts for the job and include them as allowed principals in the perimeter's access level (Access Context Manager). Configure routing so Dataflow workers resolve APIs internally - verify DNS, routes, and firewall rules allow the private paths. If you use Dataproc profiles or Flex templates, ensure the images and any initialization actions don't attempt to download artifacts from the public internet unless you've whitelisted that path inside the perimeter.

Operationalize tests and validation: run a staging job inside the perimeter and validate that it can access GCS/BigQuery/KMS/etc. without public egress. Instrument logs and VPC flow logs to verify no traffic left the perimeter and set alerts on any egress attempts. Finally, document the perimeter boundary, required resource locations, and a runbook for adding new resources or pipelines - VPC-SC imposes constraints that must be part of on-boarding and CI/CD.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

70. You're asked to implement least privilege across multiple pipelines. How would you design IAM?

Start by giving each pipeline its own service account (or a small set of service accounts per environment) instead of sharing a single account. Scope those service accounts narrowly: grant only the dataset/topic/bucket-level roles needed (e.g., bigquery.dataEditor on one dataset, storage.objectCreator on one bucket, pubsub.subscriber on one subscription). Avoid project-wide Editor/Owner bindings - prefer resource-scoped roles or custom roles that contain only the exact permissions required.

Use principle of separation by environment and role: dev/test pipelines get less sensitive rights and live in separate projects or folders. Human access follows role groups (developers, operators, release approvers) and is assigned to Google groups, not individuals. Manage changes through PRs and IaC (Terraform) so IAM is auditable and code-reviewable. Enforce approvals for any role that touches prod resources.

Create custom roles where predefined roles are too coarse, and use IAM Conditions and time-bound grants for temporary elevation (e.g., grant extra rights for a migration window). Prefer Workload Identity (GKE) or short-lived tokens rather than long-lived keys for any service that needs impersonation. For KMS access, grant only cryptoKeyEncrypterDecrypter to the specific key and only to the service accounts that need it.

Automate validation: maintain a permission manifest per pipeline (what SA needs which resource and why) and add CI checks that run smoke tests under the service account to catch missing permissions before deployment. Instrument monitoring for permission-related failures (IAM denied errors) and enforce rotation/review cadence for privileged roles. Finally, keep a central policy for provisioning new pipelines (template SAs, minimal baseline roles) so new teams inherit the least-privilege posture by default.

71. Your company wants to build an ETL from GCS → Dataflow → BigQuery. How would you design it?

I'd start by designing a clear ingestion contract: standardize file layout and naming in GCS (partitioned by date), publish a schema (Avro/Parquet/JSON schema) and a manifest mechanism so consumers know which files are complete. This lets the pipeline discover only finished files, makes reruns deterministic, and provides provenance (filename, generation, checksum) for audits and reprocessing.

Next, implement a Dataflow job that is responsible for parsing, validating and light-cleaning. The job reads files from GCS (using AvroIO/ParquetIO or TextIO for JSON), assigns event timestamps where needed, validates records (schema + business rules), emits invalid rows to a dead-letter store, and normalizes types/fields into the canonical shape expected by BigQuery. Keep transforms deterministic and idempotent so replays are safe.

For loading, prefer a staging → finalize pattern. Have Dataflow write to a partitioned staging BigQuery table or to partitioned Parquet files and then run a BigQuery load or MERGE into the production table. That lets you deduplicate, apply business upsert rules, and atomically promote partitions while avoiding expensive per-row streaming costs for large batches.

Operationalize the pipeline: parameterize inputs (GCS prefix, partition date, staging table), automate via CI/CD and an orchestrator (Cloud Composer / Cloud Scheduler), and add monitoring (file count vs rows landed, DLQ volume, job health, and end-to-end latency). Keep raw files archived and record a manifest of processed files to prevent double-processing and to support forensic replay.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Finally, add governance and testing: include schema/contract tests in CI, document the data contract and backfill/runbook, and expose a small reconciliation job that compares source file row counts and checksums to BigQuery partition counts so you can detect issues early and reprocess only what's needed.

72. A real-time fraud detection system must process Pub/Sub events in <5 seconds. How would you build it?

I'd build a low-latency streaming pipeline: Pub/Sub → Dataflow (Beam) as the processing layer because Dataflow provides autoscaling, event-time semantics, and low-latency windowing. Producers push compact, versioned protobuf/Avro events into Pub/Sub; the Dataflow job consumes, validates, and enriches events in-memory with minimal blocking operations so per-event latency stays small.

Keep the critical path extremely lightweight: do signature checks, rule-based scoring, and fast in-memory lookups (side inputs) inline. For heavier enrichments (e.g., calling external APIs), use async calls with bounded concurrency or do the expensive enrichments in a separate, non-blocking path - do not let slow external calls block the main scoring path. Use local caches (with TTL) or a low-latency store like Memorystore/Bigtable for lookups that must be fast.

Design dedupe and ordering: include deterministic event ids and use insertId or an idempotent write strategy at sinks so retries don't create duplicates. For results, push alerts to a low-latency sink (Pub/Sub alert topic, Redis, or a streaming API) and write enriched events and audit logs concurrently to BigQuery or GCS for downstream analysis and forensic replay.

Tune Dataflow for low latency: enable Streaming Engine if supported, choose appropriate worker machine types, set autoscaling thresholds for bursts, and minimize bundle sizes to reduce end-to-end per-record delay. Monitor tail latency, watermark progress, DLQ rates, and Pub/Sub ack lag; add alerts when 95/99th percentile latency approaches the 5s SLA.

Finally, validate with load testing using realistic traffic and adversarial patterns (bursts, late events, hot keys). Build a fallback / circuit-breaker strategy (e.g., degrade to rule-only scoring or sample-based processing) so the system still protects users under overload while operators triage and scale resources.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

73. You must enrich streaming clickstream data with static product catalog data. How would you implement this?

I'd treat the product catalog as a small-to-medium reference dataset and keep it versioned in a canonical location (BigQuery table, GCS JSON/Parquet, or a schema-registered source). If it's small enough to fit in memory, read it as a side input (AsDict) into the Dataflow job so each worker has a fast, in-memory lookup for enrichments; refresh the side input periodically or on change notifications to pick up updates.

If the catalog is large or updates very frequently, materialize it in a low-latency lookup store like Bigtable or Memorystore/Redis and perform async lookups from the streaming DoFn. Use asynchronous DoFns with batching and connection pooling so lookups don't block worker threads and you can parallelize requests efficiently while respecting the store's capacity and latency SLOs.

Handle schema evolution and new product attributes by keeping an extensions map or JSON column in the enriched payload; include a catalog_version or effective_ts in the side input so you can trace which catalog snapshot produced each enrichment. If enrichment correctness matters for historical reprocessing, persist raw events and the catalog snapshot id used so you can re-run enrichment deterministically later.

Operationally, monitor cache hit ratio, lookup latencies, and catalog-change events. If catalog updates are frequent and you must avoid any staleness, consider streaming catalog deltas via Pub/Sub into the pipeline and apply them to an in-memory store on workers, or use a separate pub/sub-driven update path that keeps worker caches warm without restarting jobs.

Finally, make enrichment idempotent and resilient: handle missing products gracefully (route to DLQ or enrich with defaults), expose metrics for unmatched product_ids to drive catalog quality fixes, and include unit + integration tests that exercise catalog changes and replays so both real-time and historical analytics remain consistent.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

74. IoT devices send events out of order. How would you design your pipeline?

Start by treating event-time as the source of truth - every message must carry a producer timestamp (device event time) and that timestamp should be applied in the pipeline with WithTimestamps (or equivalent). That ensures windowing and session logic are based on when the event happened, not when it was received, which is the first and most important step to get correctness with out-of-order data.

Use an event-time based windowing strategy with an appropriate watermark and allowedLateness that reflects how late devices typically arrive (e.g., 10s, minutes, or even hours). Emit early, provisional results for low-latency needs using early triggers, but keep windows open long enough to accept the usual tail of late arrivals so final aggregates are correct. Accumulating mode with idempotent upserts is the usual pattern: emit updates rather than duplicate rows.

Persist raw events immediately to an immutable archive (GCS/BigQuery changelog) before any transformation. That gives you a replay source for any corrections, backfills or changes in allowed-lateness. Couple this with a staging table and idempotent MERGE into production so replays or late-arrival reconciliations safely overwrite prior provisional results.

Handle skew and very-late outliers explicitly: monitor the distribution of lateness, and for extreme outliers either route them to a slow-path reconciliation job (manual or scheduled reprocess of that partition) or tag them for special handling (audit, compensation). Also surface metadata in outputs (watermark when emitted, pane type, version) so consumers know whether a value is provisional or final.

Finally, operationalize with metrics and runbooks: track watermark progress, late-event rates, window re-emission frequency, and SLA for finalization. Provide clear consumer expectations (e.g., “dashboards show provisional values, authoritative reports after X minutes”), and automate a replay tool that can reprocess archived raw events when you tighten/loosen lateness rules or fix logic.

75. Your team wants to migrate Spark jobs to GCP. How would you use Beam portability for this migration?

I’d start by inventorying Spark jobs: group them by complexity (simple ETL, heavy SparkSQL, custom JVM UDFs) and pick representative candidates for a proof-of-concept. Beam portability is attractive because it lets you write pipelines in Beam SDKs and run them on Dataflow (or other runners) - the migration strategy should be “lift-and-evaluate” rather than an immediate rewrite of everything.

For jobs that are essentially map/filter/aggregation or streaming logic, reimplementing in Beam (Java/Python) often maps naturally. Use Beam’s transforms (ParDo, CombinePerKey, Windowing) to express logic, and leverage Beam’s portability to run SDKs in containers on Dataflow. For SparkSQL-heavy workloads, consider porting the logic into Beam SQL where practical, or run those jobs on Dataproc (Spark) initially while migrating other pieces to Beam to reduce risk.

When you have existing Spark code with custom JVM UDFs, portability helps: either call equivalent logic in Beam Java transforms or package the UDFs into the worker container (Flex template) so Dataflow workers can run them. For polyglot teams, Beam’s cross-language transforms let you keep some parts in Python and others in Java with minimal rewriting - use that to incrementally move logic while preserving behavior.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Do a phased migration: run Beam/Dataflow pipelines in shadow mode alongside existing Spark jobs to compare outputs (row counts, checksums, sample checks). Use the raw input archive to replay and validate parity. Tune performance (parallelism, sharding, runner options) and compare cost/latency to Dataproc. If a Spark job is tightly coupled to Spark-specific optimizations that don't port well, keep it on Dataproc and migrate other workloads first.

Finally, invest in tooling and training: standardize CI/CD to build Beam artifacts or Flex images, create templates for common patterns (joins, windowing, state), and document migration playbooks. The goal is pragmatic: use Beam portability to avoid lock-in, enable unified batch+stream semantics, and migrate incrementally with measurable parity and operational controls.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar